Journal of Applied Physics TUTORIAL pubs.aip.org/aip/jap

Fine-tuning universal machine-learned interatomic potentials: A Tutorial on methods and applications

Cite as: J. Appl. Phys. **139**, 041101 (2026); doi: 10.1063/5.0299305
Submitted: 27 August 2025 · Accepted: 2 December 2025 ·
Published Online: 26 January 2026

View Online Export Citation CrossMark

Xiaoqing Liu,[1,2] Kehan Zeng,[2] Zedong Luo,[2] Yangshuai Wang,[3,a)] Teng Zhao,[2,4,b)] and Zhenli Xu[5,c)]

AFFILIATIONS

[1]School of Mathematical Sciences, Shanghai Jiao Tong University, Shanghai 200240, China
[2]Shanghai Jiao Tong University-Chongqing Institute of Artificial Intelligence, Chongqing 401329, China
[3]Department of Mathematics, National University of Singapore, 10 Lower Kent Ridge Road, Singapore
[4]Institute of Natural Sciences, Shanghai Jiao Tong University, Shanghai 200240, China
[5]School of Mathematical Sciences, MOE-LSC, CMA-Shanghai and Shanghai Center for Applied Mathematics, Shanghai Jiao Tong University, Shanghai 200240, China

[a)]**Author to whom correspondence should be addressed:** yswang@nus.edu.sg
[b)]zhaoteng_sjtu@sjtu.edu.cn
[c)]xuzl@sjtu.edu.cn

ABSTRACT

Universal machine-learned interatomic potentials (U-MLIPs) have demonstrated broad applicability across diverse atomistic systems but often require fine-tuning to achieve task-specific accuracy. While the number of available U-MLIPs and their fine-tuning applications are rapidly expanding, there remains a lack of systematic guidance on how to effectively fine-tune these models. This Tutorial provides a comprehensive, step-by-step guide to fine-tuning U-MLIPs for computational materials modeling. Using the recently released MACE-MP-0 as a representative foundation model, we illustrate the full workflow of data set preparation, hyperparameter selection, model training, and validation. Beyond methodological guidance, we conduct systematic case studies on solid-state electrolytes, stacking fault defects in metals, semiconductors, solid–liquid interfacial interactions in low-dimensional systems, and more complicated heterointerfaces. These examples demonstrate that fine-tuning substantially improves predictive accuracy while maintaining affordable computational cost, accelerates training convergence, enhances out-of-distribution generalization, and achieves superior data efficiency. Remarkably, fine-tuned foundation models can even capture aspects of long-range physics without explicit corrections. Together, these results highlight that fine-tuning not only provides a practical recipe for applying U-MLIPs but also offers new insights into their physical fidelity and potential for advancing large-scale atomistic simulations. To support practical applications, we include code examples that enable researchers, particularly those new to the field, to efficiently incorporate fine-tuned U-MLIPs into their workflows.

© 2026 Author(s). All article content, except where otherwise noted, is licensed under a Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/). https://doi.org/10.1063/5.0299305

08 February 2026 12:36:01

I. INTRODUCTION

Machine-learned interatomic potentials (MLIPs) have become an essential tool in materials modeling, offering near-density functional theory (DFT) accuracy at significantly reduced computational cost.[1–14] A broad range of MLIP frameworks have been developed in recent years, including neural network potentials,[1,4] kernel-based approaches,[2,5] and equivariant graph neural networks,[6,7] each demonstrating excellent performance across diverse materials systems. These models enable accurate and large-scale atomistic simulations, and their integration with simulation platforms such as ASE[15] and Large-scale Atomic/Molecular Massively Parallel Simulator (LAMMPS)[16] has facilitated their widespread adoption in the field. For comprehensive reviews on MLIPs, the reader is referred to Refs. 17–21.

Traditionally, most MLIPs are trained on narrowly defined chemical or structural domains tailored to specific applications. While such models can achieve high accuracy within their training scope, they often lack transferability to systems beyond that domain.

© Author(s) 2026

In response, there has been growing interest in a new class of models, often referred to as general-purpose, foundation, or universal MLIPs (U-MLIPs).[22–27] Inspired by foundation models in natural language processing and computer vision,[28] these models are pre-trained on large and diverse materials data sets,[29–31] aiming to provide broad coverage across the periodic table and structural configuration space. They offer strong baseline performance and could be readily applied to new systems with minimal additional training (i.e., fine-tuning), substantially lowering the barrier to entry for MLIP applications in materials science. In Table I, we summarize representative U-MLIPs, along with their publication year, underlying architectures, and training data characteristics.

Among existing U-MLIPs, MACE-MP-0 stands out as a near state-of-the-art model, demonstrating good accuracy and robust stability in molecular dynamics simulations across a wide range of applications.[22] Although it serves as the main example in this Tutorial, we emphasize that the methodologies and principles discussed here can be readily adapted to other common U-MLIPs (cf. Table I) with minimal modification. MACE-MP-0 is built upon the MACE architecture,[6] which integrates the atomic cluster expansion (ACE) formalism[5,43] with efficient tensor decomposition techniques[44] and higher-order equivariant message passing. Leveraging this design, MACE-MP-0 is pre-trained on a large data set from the Materials Project and achieves strong performance across diverse materials benchmarks. Several subsequent variants of the model have been introduced recently to address specific issues such as repulsion, phonons, and high-pressure stability, further broadening its applicability.

While MACE-MP-0 and its variants have demonstrated robust performance in molecular dynamics simulations,[22] its predictive accuracy, similar to that of other U-MLIPs, remains limited for certain tasks, particularly in modeling mechanical properties such as elastic constants and stacking fault energies in elemental alloys.[45] This underscores the need for fine-tuning, which adapts pre-trained models to specific systems by refining their parameters using high-fidelity computational or experimental data. Although the fine-tuning of atomistic foundation models has garnered growing attention, comprehensive and systematic studies remain limited, especially practical tutorials that guide beginners through the fine-tuning process. Recent reports have started addressing this gap,[46–54] and continued efforts are expected to clarify the best practices across various application scenarios. To the best of the authors' knowledge, only one publicly available tutorial[55] exists to date, focusing primarily on training MACE from scratch rather than providing a systematic, practice-oriented study of fine-tuning procedures, hyperparameter selection, and broad case demonstrations. This Tutorial aims to fill that gap by offering a clear, step-by-step workflow and reproducible examples designed for practical applications of fine-tuning U-MLIPs.

Beyond the pursuit of higher accuracy, a central motivation for fine-tuning in the context of MLIPs lies in a fundamental question: given a fixed data set, should one fine-tune an existing foundation model or train a new model from scratch? In practice, fine-tuning is often more efficient in terms of training time and data usage and in many cases yields comparable or even superior accuracy. While foundation models, pre-trained on large and diverse data sets, effectively capture general chemical and structural patterns, they often lack the precision required for specific systems. Fine-tuning addresses this limitation by incorporating system-specific data, thereby enhancing accuracy with significantly lower data and computational costs. Moreover, fine-tuned models typically converge faster due to the advantage of informed initialization. Therefore, when a suitable foundation model is available, fine-tuning presents a compelling alternative to training-from-scratch.

This Tutorial provides a practical guide to fine-tuning U-MLIPs, aiming to support researchers, especially those new to the field, by filling the current gap in systematic guidance for fine-tuning foundation models. The goal is to provide step-by-step instructions for preparing data, training models, selecting hyperparameters, and applying the resulting potentials in various atomistic simulations. To keep the presentation accessible, theoretical details are kept to a minimum, with references provided for readers seeking a deeper mathematical and physical understanding. The methods introduced here are demonstrated using widely used tools such as ASE, LAMMPS, and the Random Batch Molecular Dynamics (RBMD) interface.[56] MACE-MP-0 is selected as the main example due to its broad applicability, high accuracy across benchmark tasks, and demonstrated stability in molecular

08 February 2026 12:36:01

TABLE I. List of the representative U-MLIPs.

Model Name	Year	Architecture	Data Description
M3GNET[27]	2022	SchNet[32]	Materials Project (MP), 89 elements, 62 783 compounds
CHGNet[23]	2023	GNN + Charge	MP + Trajectory, 89 elements, 146 000 compounds
ALIGNN-FF[26]	2023	Line GNN	JARVIS-DFT, 72 708 compounds
PFP (Matlantis)[33]	2023	Tensorial GNN	Custom, $\approx 10\times10^6$ configurations
GNoME[24]	2023	NequIP[8]	MP + Custom, $\approx 89\times10^6$ configurations
DPA-1,2,3[25,34,35]	2023–2025	DeepMD[4]	Open-access data sets including Alloy and OC2M
MACE-MP-0[22]	2024	MACE[6]	Same data as used to build CHGNet
SevenNet-0[36]	2024	NequIP[8]	Same data as used to build CHGNet
MatterSim[37]	2024	Graph transformer	MP + Alexandria + Custom, $\approx 17\times10^6$ configurations
EquiformerV2-OMAT24[31]	2024	EquiformerV2[38]	MP + Alexandria + Custom, $\approx 118\times10^6$ configurations
NEP89[39]	2025	NEP[40]	Open-access data sets including OMat24 and SPICE
UMA[41]	2025	eSEN[42]	459M configurations; avg atoms: 19 (OMat24)–178 (ODAC25)

© Author(s) 2026

dynamics simulations.[22] We emphasize that this choice is not restrictive, and the methodologies and workflows presented in this Tutorial can be readily applied to other U-MLIPs.

Beyond methodological guidance, we conduct extensive numerical experiments that yield several key insights. Fine-tuning consistently improves predictive accuracy and accelerates convergence while maintaining affordable computational cost. It enhances out-of-distribution generalization and achieves higher data efficiency compared to training from scratch, even when using reduced data sets. It is noted that fine-tuned foundation models show improved accuracy even for problems involving long-range effects; however, this behavior is not universal and should be evaluated case-by-case. Taken together, these findings show that fine-tuning not only offers a practical recipe for applying U-MLIPs but also provides deeper insights into their physical fidelity and their potential to advance large-scale atomistic simulations.

It is worth noting that the fine-tuning of MACE-MP-0 can be efficiently carried out using the RBMD package, which enables large-scale particle simulations at the nano- and microscale. In contrast to conventional molecular dynamics frameworks, RBMD leverages random batch algorithms[57–60] to efficiently handle nonbonded interactions, supporting simulations of up to 10×10^6 particles on a single GPU with a CPU core.[56] Moving forward, we aim to integrate the RBMD platform with MACE-based MLIPs for more practical and large-scale simulations. To support this integration, an open-access platform is available at https://randbatch.com.cn/rbmd, where further examples and implementation scripts for realistic simulation scenarios will be released in the future.

The remainder of this Tutorial is organized as follows. In Sec. II, we introduce the overall fine-tuning workflow for U-MLIPs, including a brief overview of the MACE-MP-0 framework, data preparation, potential training, and model validation. Section III presents six illustrative applications of fine-tuning U-MLIPs: (i) force accuracy evaluation in the solid-state electrolyte $Li_{10}GeP_2S_{12}$ (LGPS), (ii) stacking fault energetics in body-centered-cubic molybdenum (Mo), (iii) generalization tests on out-of-distribution in semicondutor silicon, (iv) surface interactions at the graphene–water interface, (v) molecule–oxide interactions, and (vi) solid–solid interfaces, the latter two designed to assess the ability of fine-tuned models to capture long-range effects. Finally, Sec. IV provides a summary and outlook on the development and fine-tuning of U-MLIP models.

II. METHODS

In this section, we introduce the overall procedure for fine-tuning U-MLIPs, using MACE-MP-0 as a representative model. An overview of the fine-tuning workflow is shown in Fig. 1. The section is organized as follows: Sec. II A briefly describes the MACE-MP-0 model, common fine-tuning practices, and recent implementation updates. Data set construction, a critical component of MLIP development, is discussed in Sec. II B. Section II C outlines the fine-tuning process, including hyperparameter choices. Model validation is addressed in Sec. II D, which is a prerequisite for reliable molecular dynamics simulations.

The fine-tuning procedure is supported by the official MACE repository: https://github.com/ACEsuit/mace. The official MACE

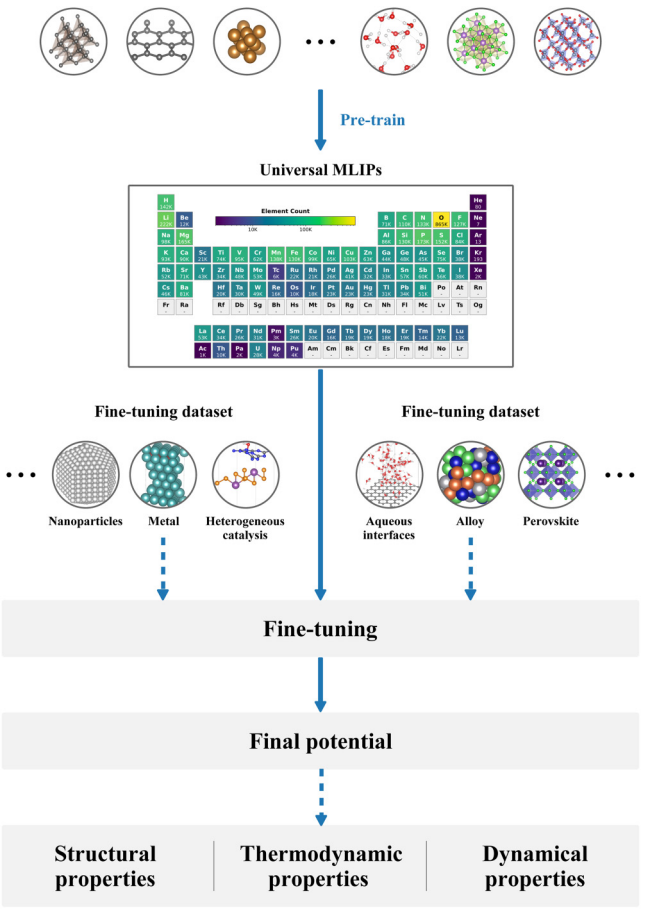


FIG. 1. Overview of the fine-tuning workflow.

documentation (https://mace-docs.readthedocs.io/en/latest/) offers up-to-date recommendations and usage guidelines. Installation instructions are also available in the tutorial repository: https://github.com/John2021-hub/mace-ft-tutorial.git. The RBMD interface[56] supports both fine-tuning and molecular dynamics simulations with MACE-based MLIPs.

A. The MACE-MP-0 model: Architecture, fine-tuning, and recent developments

We first briefly introduce the MACE-MP-0 model in Sec. II A 1, and detailed description is provided in the Appendix. The fine-tuning procedure introduced here is broadly applicable to other U-MLIPs with minor adjustments.

1. Overview of the MACE-MP-0 model

Before introducing U-MLIPs, we briefly recall the concept of MLIPs. These models map atomic positions and chemical elements to the potential energy of a given atomic system. The MACE architecture[6] is a representative example, extending the Atomic Cluster

08 February 2026 12:36:01

© Author(s) 2026

Expansion (ACE) framework,[5,43,61] which was originally developed for MLIPs and has since found applications beyond atomistic modeling.[62–65] MACE is an equivariant message-passing graph tensor network in which each layer directly encodes many-body correlations via tensor products of atom-centered features, rather than building many-body effects solely by stacking two-body layers.[8] Messages are built from linear combinations of tensor product bases derived from two-body permutation-invariant polynomials in a spherical basis.[5,43] Equivariance is maintained through tensor contractions with irreducible representations and generalized Clebsch–Gordan coefficients.[44,66] The final output represents the per-atom contribution to the total potential energy. Further architectural details are provided in Ref. 6.

The foundation model MACE-MP-0, trained on the Materials Project data set,[67] has demonstrated strong performance in a range of applications,[22] particularly in enabling stable molecular dynamics simulations across diverse chemical environments. It is released in three variants, differentiated by increasing levels of message-passing equivariance ($L = 0, 1, 2$). A subsequent revision, MACE-MP-0b, introduced minor improvements such as refined pairwise repulsion and improved treatment of isolated atoms. Further variants have been developed to address specific limitations: MACE-MP-0b2 enhances stability under high pressure, MACE-MP-0b3 corrects phonon behavior, and MACE-MPA-0 and MACE-OMAT-0 extend training coverage. All models are publicly available at https://github.com/ACEsuit/mace-mp/. In this work, we test these variants as released (cf. Sec. III) and explicitly refer to each model by its full name whenever it appears.

2. Fine-tuning strategies

Fine-tuning pre-trained models is widely adopted to improve accuracy and transferability in various domains, including image analysis[68,69] and natural language processing.[70–72] In molecular modeling, fine-tuning typically follows a predictor–corrector paradigm: a U-MLIP serves as an initial predictor that provides a robust baseline for atomistic simulations. The model is then fine-tuned on a system-specific data set, allowing it to correct and adapt to the target system with improved accuracy.

Multi-head replay fine-tuning extends this idea by attaching multiple task-specific output heads to a shared pre-trained backbone.[73] Each head is tailored to a different target system or property, while the backbone captures general representations. In the default MACE implementation, the backbone remains trainable during multi-head fine-tuning. Partial freezing is optional and may be used for efficiency or regularization.[74] This design also mitigates catastrophic forgetting by preserving the performance of previously trained heads while adapting to new tasks. An implementation of this method is available at https://github.com/ACEsuit/mace/tree/main.

From an optimization perspective, existing fine-tuning strategies for U-MLIPs can be broadly categorized into (i) full-parameter fine-tuning, where all backbone and head parameters are updated.[75] This maximizes task-specific adaptability but incurs higher computational cost and greater risk of overfitting, especially when the target data set is small. (ii) Partial freezing, where early layers or invariant feature extractors are kept fixed, while only later layers or heads are updated. This reduces the number of trainable parameters, leading to faster convergence and better knowledge retention from the pre-trained model, but may limit adaptation to systems with strong distribution shifts.[74] (iii) Lightweight adaptation layers (e.g., low-rank adapters or bias tuning), which introduce a small set of trainable parameters on top of the frozen backbone.[76] This approach offers a favorable trade-off between efficiency and accuracy and has shown promise in retaining generalization while improving data efficiency.

In molecular modeling, the choice of strategy should be guided by the similarity between the pre-training and target domains, the size and diversity of the fine-tuning data set, and the computational budget. A comprehensive benchmark to determine the optimal strategy under different scenarios is still lacking, but future work will aim to provide such guidance.

3. Recent updates

This section briefly introduces two recent updates relevant to U-MLIP deployment: the fast equivariant kernel cuEquivariance and an implementation of long-range dispersion corrections with efficient neighbor list handling. These tools are integrated into our examples (cf. Sec. III) to demonstrate the fine-tuning and application of the MACE-MP-0 model in practical simulations.

1. The fast equivariant kernel cuEquivariance. Equivariance, referring to the preservation of pre-defined physical symmetry under group operations such as rotations and translations, is a key property of physically meaningful interatomic potentials. Models that are naturally equivariant are typically more data-efficient, possess better physicality, and exhibit improved generalization.

cuEquivariance is an NVIDIA-developed library for building high-performance equivariant neural networks based on segmented tensor products within easily accessible Python interface. It provides a flexible API and optimized CUDA kernels, with bindings for both PyTorch and JAX. The code is available at https://github.com/NVIDIA/cuEquivariance.

Fine-tuned MACE-MP-0 models (cf. Sec. II C) can be integrated with cuEquivariance in Python-based simulation environments such as ASE,[15] enabling significantly faster inference, typically by a factor of 3–10. Example usage and performance benchmarks are provided in Sec. III A. The following code snippet illustrates how to activate the fast kernel via a simple keyword:

```
from mace.calculators import MACECalculator
mace_calc =
    MACECalculator(model_paths="MACE.model",
    enable_cueq=True, device="cuda")
```

As reported by developers, the jax implementation is expected to have at least extra few factors of speed up further.

2. Long-range dispersion corrections with efficient neighbor list handling. Frequent neighbor list updates can introduce significant computational overhead, particularly when large cutoffs are used for dispersion corrections such as DFT-D3.[77] To mitigate this, a simple yet effective neighbor list implementation with a skin layer is provided in ASE. This approach reduces the frequency of

08 February 2026 12:36:01

© Author(s) 2026

neighbor list reconstruction and improves overall computational efficiency.

The implementation allows for a similar update strategy in LAMMPS and enables the reuse of neighbor lists across multiple simulation steps. An example configuration is shown below:

```
calc = TorchDFTD3Calculator(atoms=atoms,
    device="cuda", damping="bj",
    every=2, delay=10, check=True, skin=2.0)
```

In this example, the parameters every, delay, check, and skin control the update frequency and reuse behavior of the neighbor list:

- every: interval between neighbor list updates.
- delay: buffer before enforcing the next update.
- check: whether to validate the reuse condition.
- skin: width of the buffer zone (in Å), allowing small displacements without triggering a rebuild.

The implementation is available at https://github.com/CheukHinHoJerry/torch-dftd.

B. Data preparation

The quality of a MLIP is closely tied to the quality of the reference data set. MLIPs are typically trained via supervised regression on a large set of atomic configurations, each labeled with total energies, atomic forces, and often stress tensors, usually computed from DFT. The same principles apply to fine-tuning U-MLIPs, where a well-designed data set is crucial for achieving improved accuracy and transferability.[46,48,49,78,79]

1. Empirical data set construction

Empirical data set construction relies on physically motivated or experience-driven methods. These include using existing public data sets, generating configurations via random perturbations, or sampling configurations from molecular dynamics (MD) or Monte Carlo (MC) simulations. Each method offers distinct advantages and is best suited for particular types of systems or user expertise. In the following, we briefly describe each approach and offer practical recommendations based on our experience.

1. Using publicly available data sets. Depending on observables or quantities of the target system, open-access data sets that have been developed for machine-learned interatomic potentials can be readily used for fine-tuning U-MLIPs.[29–31,80–82] A key practical issue is format compatibility. For example, MACE-MP-0 expects extended .xyz input, whereas Deep Potential (DP) data sets[4] are often in .npy. A ready-to-use converter, together with usage notes and tests, is provided in the tutorial repository.

Another consideration is data set size. Public data sets designed for training-from-scratch are often much larger than what is needed for fine-tuning. Since U-MLIPs already encode broad chemical and structural trends, effective fine-tuning can usually be achieved with significantly fewer configurations. This observation will be demonstrated in a case study in Sec. III A. We note that using publicly available data sets is especially recommended for beginners as it requires minimal setup and provides immediate access to high-quality reference data.

2. Random perturbation of structures. Another common approach is to apply random displacements to atoms in relaxed structures to sample configurations near local energy minima (local equilibrium). This method is simple and widely used, especially for systems with crystalline structures. If stress data are required, small random deformations of the simulation cell (i.e., strain perturbations) should also be applied. Random perturbations can be generated using ASE's rattle function, typically with Gaussian-distributed displacements of 0.1–1.0 Å. Care must be taken to avoid unphysical structures with atoms too close together. This issue can be mitigated by applying a Monte Carlo filtering procedure that enforces a minimum interatomic distance, where configurations with excessively short atomic separations are accepted with low probability.[83] This method is recommended for users who are familiar with atomistic simulation workflows and have sufficient computational resources. It is particularly well suited for advanced users with prior knowledge of the target system. Notably, when combined with the filtering techniques discussed later, well-designed perturbation strategies can produce compact and physically meaningful data sets that are particularly effective for fine-tuning U-MLIPs.[84]

3. Sampling from MD or MC simulations. When no well-defined structural prototype exists, as in amorphous phases or liquids, perturbation methods are not suitable. Instead, *ab initio* molecular dynamics[85,86] or Monte Carlo simulations provide a more general approach for generating configurations. Snapshots are extracted at regular intervals along the trajectory to build the data set. Although computationally more demanding, this approach is essential when structural templates are unavailable. In practice, a widely used two-stage *bootstrapping* workflow is often more efficient: first, generate initial trajectories with empirical or mixed-level force fields (e.g., classical MD in LAMMPS) to explore the configurational space and pre-equilibrate the system under physically consistent thermodynamic conditions and second, *upgrade* a subset of frames by relabeling them with higher-fidelity DFT, guided by uncertainty or diversity criteria. This procedure can be iterated if needed. We follow this workflow for the graphene–water example in Sec. III D. To further improve data set diversity and relevance for fine-tuning, perturbations are often applied to MD-generated structures; however, care must be taken to avoid unphysical configurations, as excessive distortions can lead to unrealistic atomic overlaps or unstable geometries. Additionally, MD simulations with enhanced sampling could be employed.[87,88]

2. Uncertainty-based data set construction

Manual data set design often depends on expert intuition, which may introduce inductive biases, redundant structures, or sampling imbalances. Such issues can undermine data efficiency and hinder the generalizability of trained MLIPs. As a remedy, uncertainty-guided data selection has emerged as a systematic and

08 February 2026 12:36:01

© Author(s) 2026

efficient alternative for both training MLIPs and fine-tuning U-MLIPs.[89–94]

Uncertainty quantification (UQ) estimates the confidence of a model's predictions and can be used to identify regions in the configuration space where the model is unreliable. Incorporating such estimators enables active-learning strategies that prioritize configurations with high predictive uncertainty, thereby improving accuracy while minimizing data redundancy.

Several definitions and methodologies for uncertainty estimation are commonly employed in the context of MLIPs. These approaches can be broadly categorized into frequentist and Bayesian frameworks, as summarized in Table II. Frequentist techniques include model ensembles[95] and feature-space distance metrics,[96] while Bayesian methods rely on posterior inference,[92] dropout-based approximations,[93] or analytical forms available in tractable models such as Gaussian processes and linear regressions.[84] In addition, hybrid approaches have been developed to separately quantify aleatoric (data-inherent) and epistemic (model-related) uncertainties.[97] A comprehensive discussion of UQ in MLIPs is beyond the scope of this Tutorial. However, we emphasize that this remains an active area of research, and we refer interested readers to recent reviews for further insight.[98–100]

1. Filtering a candidate set. In cases where a large pool of configurations is available (e.g., from long MD simulations or existing data sets), uncertainty-based filtering can be applied to construct an efficient fine-tuning set. Here, an uncertainty estimator (such as an ensemble or a surrogate model with Bayesian regression) is used to assign a confidence value to each configuration. Configurations with higher estimated uncertainty are prioritized, as they are more likely to contribute new information to the model, while unphysical configurations are naturally filtered out by this procedure. This strategy has been used successfully in recent works, such as Ref. 84, where a linear ACE model is trained on top of an existing representation to guide uncertainty filtering. We evaluate the data efficiency of fine-tuning by comparing it against uncertainty-based filtering in Sec. III B.

TABLE II. Representative uncertainty quantification (UQ) methods used in MLIPs.

Method	Key idea	Category
Ensemble[101]	Use variance across multiple independently trained models	Frequentist
Dropout UQ[102]	Dropout at inference to sample pseudo-ensembles	Bayesian
Bayesian NN[103]	Infer posterior over weights via VI[104] or MCMC[105]	Bayesian
Evidential UQ[106]	Predict mean and variance directly as outputs	Evidential
GPR variance[107]	Analytical uncertainty from Gaussian processes	Bayesian
Feature distance[108]	Distance to the nearest training point in feature space	Geometric
Aleatoric + Epistemic[109]	Explicitly model data and model uncertainty	Combined

2. Active-learning strategy. Active learning provides a fully iterative framework for data set construction. Starting from a small initial set of labeled configurations, the MLIP is trained and then used to run MD simulations. Configurations from these simulations are evaluated by a UQ metric, and the most uncertain ones are selected for DFT labeling. The newly labeled data are added to the training set, and the process is repeated until convergence or a stopping criterion is met (e.g., error saturation or maximum data budget). This strategy is particularly effective for exploring diverse chemical spaces or unknown physical regimes. However, when applied continuously to fine-tune foundation models, active learning must be used with care, as iterative retraining without proper safeguards can lead to *catastrophic forgetting*, where previously learned knowledge is partially lost. Understanding and mitigating this effect is an important direction for future research on active-learning-based fine-tuning of foundation models.

While uncertainty-driven methods offer significant advantages, they are not universally applicable. Analytical UQ is often limited to linear models or kernel-based methods such as Gaussian process regression. For large, non-linear models like deep neural networks, approximate UQ methods (e.g., ensembles, dropout) may require careful calibration.[110,111] Additionally, active-learning workflows introduce algorithmic and software complexity, which may be a barrier for new users.

For novice users, we recommend the following practical guidelines: (i) small, chemically narrow systems: start with a simple ensemble-based UQ (e.g., multiple MACE models trained from different random seeds) and select new configurations with the highest predicted force uncertainty. (ii) Chemically diverse or poorly explored systems: consider dropout-based UQ during inference to reduce training cost while still capturing model variance but apply a post-hoc calibration step such as isotonic regression or temperature scaling. (iii) When DFT labeling is expensive, limit the number of configurations per iteration by combining UQ-driven selection with structural diversity filters (e.g., farthest-point sampling in descriptor space[112]). These approaches balance exploration efficiency, calibration accuracy, and computational feasibility, enabling active-learning workflows that are both effective and accessible to new users.

C. Fine-tuning

This section outlines the main steps and considerations for fine-tuning the MACE-MP-0 model. Fine-tuning refers to updating the parameters of a pre-trained U-MLIP on a task-specific data set, while keeping the overall model architecture fixed. This strategy leverages the broad physical knowledge encoded in the foundation model and enables efficient adaptation to new systems with relatively limited data.

A typical fine-tuning workflow involves constructing the training data set (and usually a validation data set) set with the above mentioned data set construction methods. Table III summarizes the representative hyperparameters and their recommended ranges, consistent with the example script provided in Listing 1.

08 February 2026 12:36:01

© Author(s) 2026

TABLE III. Representative hyperparameters for fine-tuning MACE-MP-0. The recommended values are representative for the cases considered in this work and may require adjustment for different systems.

Parameter	Description	Recommended range
energy_weight	Energy loss weight	1.0–20.0
forces_weight	Force loss weight	1.0–20.0
stress_weight	Stress loss weight	0.0–20.0
lr	Learning rate	10^{-4}–10^{-3}
batch_size	Batch size	2–20
ema_decay	EMA decay rate	0.98–0.999 99
swa	Enable stochastic weight averaging	True/False
swa_lr	Learning rate for SWA phase	10^{-5}–10^{-4}

Listing 1 presents a fine-tuning script using the run_train.py utility provided by the MACE package:

```
python3 $MACE_DIR/mace/cli/run_train.py \
    --name=$RES_DIR \
    --foundation_model="XXX.model" \
    --model_dir=results/$RES_DIR \
    --log_dir=results/$RES_DIR \
    --checkpoints_dir=results/$RES_DIR \
    --results_dir=results/$RES_DIR \
    --train_file="train.xyz" \
    --valid_file="valid.xyz" \
    --energy_weight=1.0 \
    --forces_weight=10.0 \
    --stress_weight=1.0 \
    --loss="universal" \
    --forces_key=dft_force \
    --energy_key=dft_energy \
    --stress_key=dft_stress \
    --lr=0.0005 \
    --scaling="rms_forces_scaling" \
    --batch_size=4 \
    --max_num_epochs=150 \
    --ema \
    --ema_decay=0.99 \
    --weight_decay=1e-6 \
    --amsgrad \
    --default_dtype="float64" \
    --clip_grad=10 \
    --device=cuda \
    --seed=$SEED \
    --num_samples_pt=500 \
    --stage_two_energy_weight=100.0 \
    --stage_two_forces_weight=10.0 \
    --stage_two_stress_weight=1.0 \
    --stage_two \
    --stage_two_lr=5e-4 \
    --E0s="{3:-0.29745415, 15:-1.88719893,
        16:-1.08090657, 32:-0.77926738}" \
```

From our experience, a medium-sized model ($L = 1$) provides the most balanced trade-off between accuracy and computational efficiency. While the small model ($L = 0$) is computationally inexpensive but lacks expressivity, the large model ($L = 2$) increases accuracy at the cost of significantly reduced efficiency. The following recommendations are in general helpful on stability and accuracy of the resulting model from fine-tuning:

- Isolated atom energies (E0s): It is very important to compute your own E0s, which specify the isolated atom energy as a dictionary in your DFT calculation rather by estimating it from least square regression with E0s="average". When computing your E0s, use spin polarized calculations. You may also use E0s="foundation" to use the same E0s when using exactly the same DFT settings and software as Materials Project.
- Learning rate (lr): If the training or validation loss stagnates or diverges, consider modifying the learning rate from recommended range for more stable convergence.
- Loss weights: Depending on the target properties, modify the ratio between energy, forces, and stress weight in fine-tuning within the recommended range.
- Cutoff radius (r_max): Do not modify the pre-defined cutoff radius that the foundation model used.
- Batch size: A larger batch size generally improves training stability but requires more GPU memory. Batch size larger than 20 is not recommended since a larger batch size, in general, tends to deteriorate the generalizability of machine learning models.
- EMA (ema) and stage two (stage_two): Enable the exponential moving average (EMA) to smooth optimization and improve generalization. Use stage_two to upweight the energy loss in the final training phase and improve energy accuracy. If training dynamics appear unstable, set ema_decay to a high value (e.g., 0.999 99). The model with the lowest validation loss, whether from stage 1 or stage 2 (cf. Fig. 2), is saved and subsequently validated on downstream properties.
- Number of samples in the pretrained head (num_samples_pt): This parameter controls how many samples from the pretrained model's output distribution are retained for the fine-tuning stage. A larger value preserves a richer set of features and variance from the pretrained head, which can improve MD stability. In other words, it acts as a form of "knowledge retention" that mitigates catastrophic forgetting of the pretrained model. However, there is a trade-off: if it is too large, the fine-tuned model remains overly biased toward the pretrained distribution, which may limit its ability to fit task-specific data and, thus, cap the achievable accuracy.

We recommend monitoring convergence behavior (cf. Fig. 2) and validation accuracy throughout the fine-tuning process. Evaluating model performance on unseen configurations is crucial and will be discussed in Sec. II D.

D. Model validation and evaluation

After completing the fine-tuning process, several output files are generated by the training script. Among them, the most critical is the log file, which contains all essential information needed to reproduce the results. This includes the selected hyperparameters, loss trajectories over all training epochs, and final test errors (typically reported in terms of RMSE). Toward the end of the log, a summary table presents the test errors for energies, forces, and, if available, stresses.

Additionally, the script automatically generates diagnostic plots that visualize the training and validation loss curves, as well as scatterplots comparing predicted and reference energies and

08 February 2026 12:36:01

© Author(s) 2026

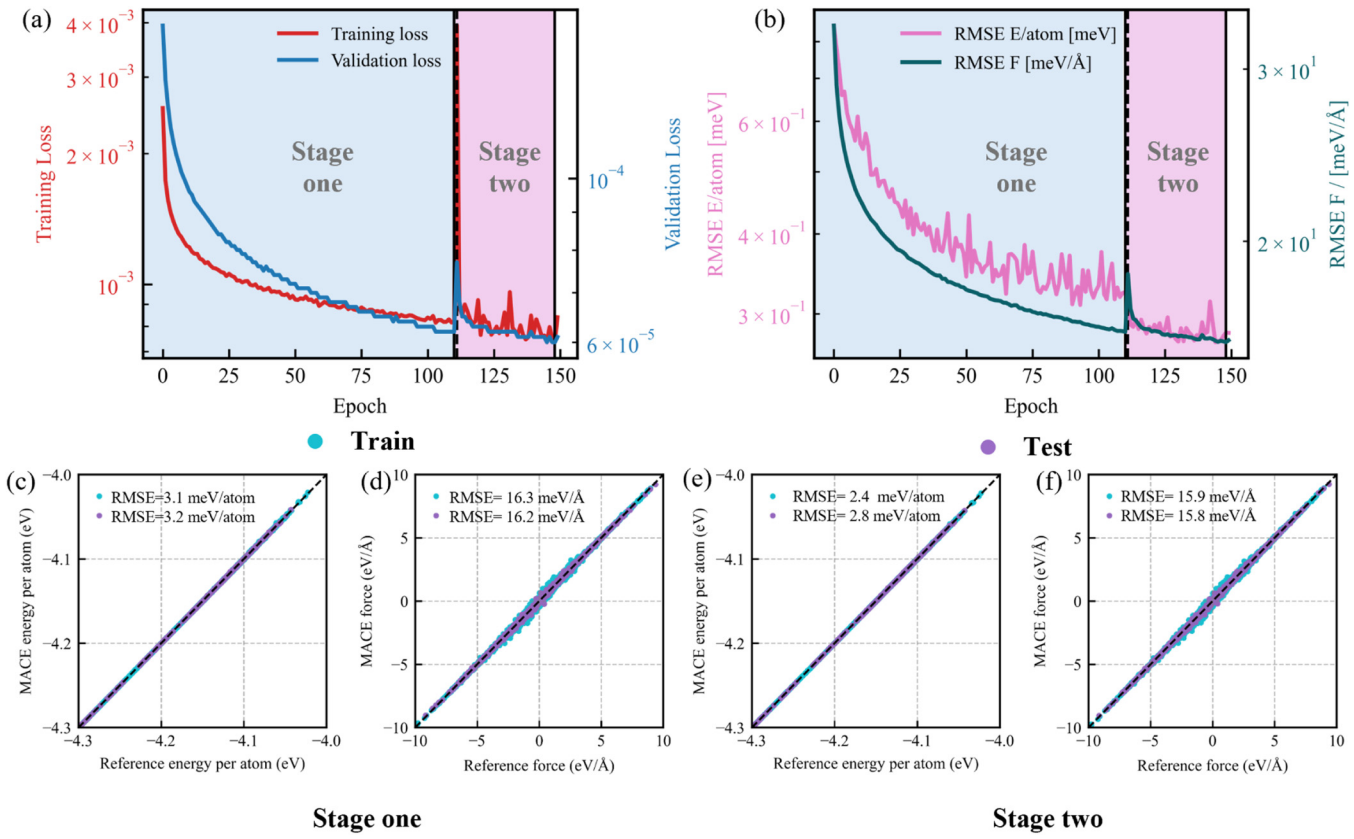


FIG. 2. Training and validation performance of fine-tuning. (a) Training and validation loss curves for stage 1 and stage 2, where the energy weighting relative to forces is increased. (b) Validation RMSE for energies and forces across epochs in both stages. (c)–(f) Parity plots comparing predicted and reference energies (c) and (e) and forces (d) and (f) for stage 1 and stage 2, respectively.

forces. These visualizations provide a qualitative assessment of model performance. In particular, smooth loss convergence and tightly clustered scatterplots are indicative of successful fine-tuning.

As an illustration, we refer to the example in Sec. III A, where such diagnostic plots are used to evaluate the performance of a fine-tuned model. Figure 2 displays the evolution of training and validation losses, along with the final energy and force scatterplots.

The output directory also contains several key model files:

- MACE-FT.model: the complete fine-tuned model used for inference;
- MACE-FT_compiled.model: a compiled version optimized for deployment; and
- MACE-FT_stageone.model and MACE-FT_stagetwo.model: intermediate models from different training stages (e.g., before and after stochastic weight averaging).

These models can be directly used in downstream atomistic simulations and analysis workflows to evaluate a variety of important material properties. These include structural and mechanical properties such as bulk modulus and defect formation and migration energies, thermodynamic properties such as density, pressure, and phonon spectra, as well as dynamical properties such as radial distribution functions and diffusion coefficients obtained from molecular dynamics simulations.

3. Example: Structural relaxation with ASE. Once fine-tuned, the model can be seamlessly integrated into the Atomic Simulation Environment (ASE) for structural optimization and property evaluation. The example below demonstrates how to perform structural relaxation on a given atomic configuration using the MACECalculator within ASE:

```
from mace.calculators import MACECalculator
ase_atoms.calc = MACECalculator("MACE-FT.model")

from ase.optimize.precon import PreconLBFGS
optimizer = PreconLBFGS(ase_atoms,
    variable_cell=True)
optimizer.run(fmax=1e-5)
```

This procedure minimizes both atomic positions and cell parameters using the preconditioned L-BFGS algorithm. The same workflow can be extended to defect formation energy calculations,

08 February 2026 12:36:01

© Author(s) 2026

nudged elastic band (NEB) transition state searches, or molecular dynamics (MD) simulations.

4. Example: Using the model in LAMMPS. To use the fine-tuned model in LAMMPS, it must first be converted to a LAMMPS-compatible format using the provided utility script:

```
python
    <mace_repo_dir>/mace/cli/create_lammps_model.py
    MACE-FT.model
```

Executing the above command generates a LAMMPS-compatible model file named MACE-FT.model-lammps.pt, which can be directly used in LAMMPS simulations via the following commands:

```
pair_style    mace no_domain_decomposition
pair_coeff    * *  MACE-FT.model-lammps.pt elmemt1
    elment2 elment3
```

Here, pair_style specifies the MACE interaction style without domain decomposition, and pair_coeff is used to load the compiled model file along with the corresponding atomic species. In our molecular dynamics simulations within LAMMPS, we adopted the metal unit style and applied a canonical (NVT) ensemble to maintain the temperature at 298 K. It is important to ensure consistency in unit settings across different simulation components, as mixing unit styles can lead to physically incorrect results.

We refer readers to our third case study (Sec. III D) for a complete example of running MD simulations in LAMMPS using a fine-tuned model. All input files and scripts are available in our public tutorial repository: https://github.com/John2021-hub/mace-ft-tutorial.git.

III. EXAMPLES AND APPLICATIONS

The fine-tuned models integrate seamlessly with widely used atomistic simulation platforms such as ASE, LAMMPS, and RBMD (see the Appendix), enabling efficient molecular dynamics and other atomistic simulations. In this section, we illustrate the effectiveness of fine-tuning using representative case studies on solid-state electrolytes, metallic stacking faults, and interfacial phenomena in low-dimensional systems. The pretrained foundation models are publicly available at https://github.com/ACEsuit/mace-mp/. In addition, we provide fine-tuning scripts, example outputs, and best practices in our tutorial repository: https://github.com/John2021-hub/mace-ft-tutorial.git.

We performed fine-tuning and molecular dynamics simulations on a single NVIDIA A800 GPU with 80 GB of HBM memory. The software environment was configured with CUDA 11.7 and the NVIDIA HPC SDK (nvhpc) 20.9. Unless otherwise specified, all training runs were carried out under this single-GPU setup.

We present four representative examples to illustrate the fine-tuning of U-MLIPs across different materials contexts. Solid-state electrolytes are chosen to assess improvements in force prediction essential for accurate ion-transport simulations. Stacking fault defects in metals are included to evaluate generalization to defect configurations and the accurate prediction of stacking fault energies. Silicon with a diverse and complex data set is used to evaluate out-of-distribution generalization. Solid–liquid interfacial systems are examined to test the model's ability to capture interfacial energetics critical to phase transitions and growth processes.

A. $Li_{10}GeP_2S_{12}$

In this example, we demonstrate the fine-tuning procedure on a solid-state electrolyte system, lithium germanium phosphorous sulfide ($Li_{10}GeP_2S_{12}$, abbreviated as LGPS), using a data set published on DPAISquare.[80] LGPS is a prototypical member of the LGPS-type thiophosphate family and has attracted significant attention due to its high lithium-ion conductivity, making it a promising candidate for next-generation all-solid-state batteries. Modeling such materials requires accurate interatomic potentials capable of describing both local structural features and ionic transport mechanisms, making LGPS an ideal testbed for evaluating the transferability and fine-tuning of U-MLIPs.

The LGPS data set published on DPAISquare[80] can be converted into the .xyz format supported by MACE, making it directly usable in the fine-tuning workflow (cf. Listing 1). Figure 3 shows the atomic structure of a typical configuration from this data set.

Table IV summarizes the test accuracy of multiple MACE foundation models after fine-tuning. All models achieve comparable accuracy in terms of root mean square error (RMSE) with respect to DFT reference values. The two rows for each model correspond to results obtained from stage 1 and stage 2, where the first epoch indicates the SWA start step, and the second corresponds to the final averaged model. The consistency of test accuracy before and after SWA highlights the stability and robustness of the fine-tuning procedure. In addition, the key point is that for all U-MLIPs, with the same hyperparameters, we can all get comparable accuracy, showing the robustness of MACE-based foundation models and its fine-tuning.

Figure 4 compares the test accuracy of models fine-tuned from different MACE foundation models with that of training-from-scratch,

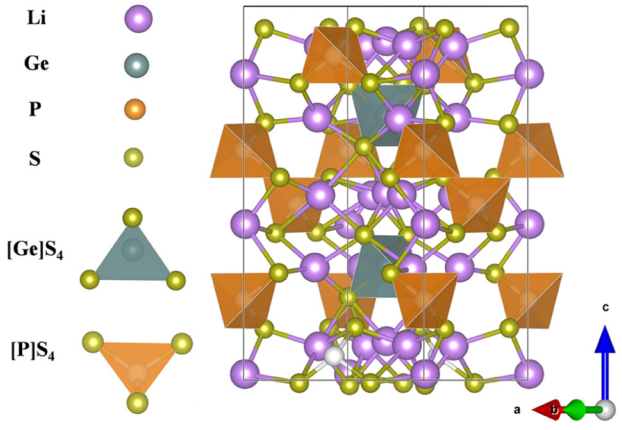


FIG. 3. Atomic structure of $Li_{10}GeP_2S_{12}$, a solid-state electrolyte with high lithium-ion conductivity.

08 February 2026 12:36:01

© Author(s) 2026

TABLE IV. Test accuracy of different fine-tuned MACE foundation models, reported as the RMSE of energy (E) and force (F), relative to DFT values. "Relative F" denotes the force RMSE normalized by the standard deviation of DFT forces. Each model is evaluated with the epoch number indicating the training step (stage 1 and stage 2). Fine-tuning consistently achieves good accuracy regardless of which foundation model is used.

Model (epoch)	RMSE E (meV/atom)	RMSE F (meV/Å)	Relative F (%)
MACE-MP-0b3 (110)	0.32	16.15	1.92
MACE-MP-0b3 (148)	0.28	15.84	1.87
MACE-MPA-0 (110)	0.30	15.07	1.79
MACE-MPA-0 (146)	0.27	14.88	1.77
MACE-OMAT-0 (110)	0.35	15.46	1.84
MACE-OMAT-0 (149)	0.27	15.32	1.82

evaluated at different fractions of the training data set (10%, 30%, 50%, 75%, and 100%). Figure 4(a) shows the RMSE in energy, while Fig. 4(b) shows the RMSE in forces. Across all data set sizes, fine-tuned models (MP-0b3, MPA-0, and OMAT-0) consistently achieve lower errors than training-from-scratch, particularly at lower data fractions, demonstrating the data efficiency and accuracy benefits of fine-tuning. The error bars remain small across experiments, indicating robustness against training noise. Overall, these results highlight the strength of MACE foundation models as effective starting points for fine-tuning, enabling improved performance even with limited training data.

To assess the influence of key training hyperparameters on model performance, a systematic ablation study was conducted for three representative quantities: force weighting (forces_weight), learning rate (lr), and EMA decay rate (ema_decay). The corresponding results are summarized in Table V. Within the ranges tested, the combination of `forces_weight` = 100, `lr` = 0.001 and `ema_decay` = 0.999 consistently yielded the lowest RMSE for force predictions, while the energy RMSE remained essentially unchanged. These findings indicate that appropriate hyperparameter tuning can enhance accuracy and training stability. Nevertheless, identifying optimal values is both nontrivial and computationally demanding. Future work will focus on developing an automated and systematic workflow for hyperparameter optimization to further improve the efficiency and reproducibility of fine-tuning.

TABLE V. Ablation study of hyperparameter choices and their influence on the root mean square error (RMSE) of energy and force predictions for the LGPS data set. Each value represents the mean over three independent runs with different random seeds; values in parentheses indicate the corresponding standard deviation. Bold entries highlight the best-performing hyperparameter settings within each group.

Hyperparameter	Value	RMSE E (meV/atom)	RMSE F (meV/Å)	Relative F (%)
Force weight (forces_weight)	1.0	0.33 (0.01)	18.71 (0.13)	2.22 (0.02)
	10.0	0.28 (0.01)	15.80 (0.02)	1.88 (0.01)
	20.0	0.27 (0.01)	15.55 (0.01)	1.85 (0.01)
	50.0	**0.26 (0.01)**	15.11 (0.03)	1.83 (0.06)
	100.0	0.27 (0.01)	**14.80 (0.03)**	**1.76 (0.01)**
Learning rate (lr)	0.0001	0.33 (0.01)	18.52 (0.08)	2.20 (0.01)
	0.0005	0.27 (0.01)	15.76 (0.03)	1.87 (0.01)
	0.001	**0.26 (0.01)**	**15.24 (0.02)**	**1.81 (0.01)**
	0.005	0.28 (0.01)	16.14 (0.04)	1.92 (0.01)
	0.01	0.29 (0.01)	17.23 (0.13)	2.05 (0.02)
EMA decay (ema_decay)	0.95	0.28 (0.01)	15.88 (0.03)	1.89 (0.01)
	0.98	0.28 (0.01)	15.81 (0.01)	1.88 (0.01)
	0.99	0.28 (0.01)	15.77 (0.02)	1.87 (0.01)
	0.999	**0.27 (0.01)**	**15.70** (0.01)	**1.87 (0.01)**
	0.9999	0.31 (0.02)	15.78 (0.02)	1.91 (0.05)

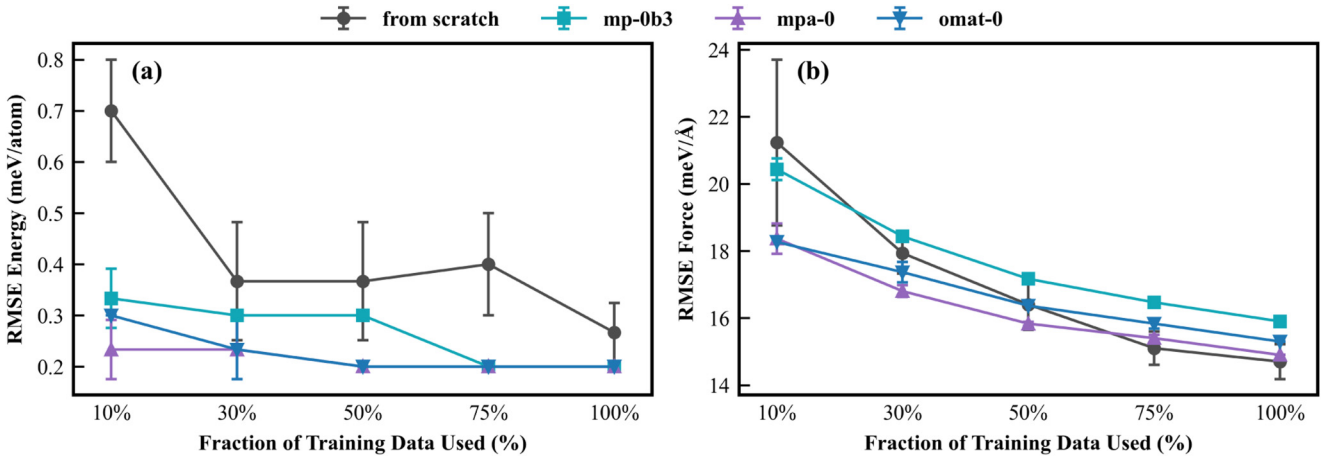


FIG. 4. Comparison between fine-tuning and training-from-scratch in terms of data efficiency: (a) RMSE in energy (meV/atom) and (b) RMSE in forces (meV/Å). Error bars represent one standard deviation, computed from three independent runs, with markers showing the corresponding mean values.

08 February 2026 12:36:01

© Author(s) 2026

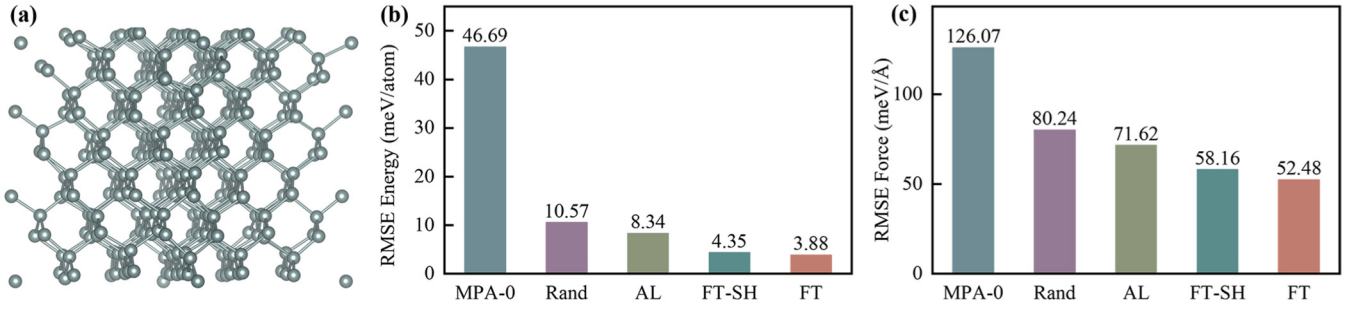


FIG. 5. Comparison between the foundation model (MACE-MPA-0) and its fine-tuned variants on the silicon out-of-distribution test set. (a) Atomic structure. (b) Energy RMSE per atom (meV/atom). (c) Force RMSE (meV/Å). MACE-FT achieves substantially lower errors than MACE-MPA-0. MACE-FT also outperforms models trained on uncertainty-filtered data (AL) and randomly selected data (Rand) and is slightly better than the single-head fine-tuning variant (FT-SH).

B. Silicon

We use silicon studied in Ref. 113 as a representative system to evaluate the out-of-distribution generalization of fine-tuning, as the potential risk of overfitting has attracted considerable attention in the fine-tuning of foundation models. The training set consists of standard bulk and defect configurations, while the test set includes grain boundaries, di-interstitials, stacking fault paths, and an amorphous configuration. Unlike a random split, the test set contains configurations entirely distinct from the training data, providing a stringent benchmark to assess extrapolative performance.

The previous example highlighted the data efficiency of fine-tuning with limited training data. In this case, we focus on active learning based on ensemble-uncertainty, where candidate configurations are filtered according to prediction uncertainty (cf. Sec. II B 2) following the procedure in Ref. 84. This reduces the training set to only 5% of the original data. We then train a MACE model on this reduced data set and compare its accuracy with that obtained from direct fine-tuning.

Figure 5 compares the performance of the foundation model (MACE-MPA-0) and its fine-tuned counterpart (MACE-FT) on the silicon out-of-distribution test set: (a) energy RMSE per atom (meV/atom) and (b) force RMSE (meV/Å). Across both metrics, MACE-FT achieves markedly lower errors than MACE-MPA-0, indicating improved agreement with the reference data and confirming that fine-tuning maintains, rather than degrades, generalization capability. Furthermore, compared to models trained on uncertainty-filtered data sets (labeled "AL" in the figure), MACE-FT achieves superior accuracy even with a comparable number of samples. This shows that fine-tuning offers better data efficiency than training from scratch on uncertainty-driven subsets. The "AL" models, in turn, outperform those trained on randomly selected data ("Rand"), highlighting the value of uncertainty-based data selection.

Last, we evaluate the effect of multi-head fine-tuning. In this out-of-distribution test, the multi-head variant (MACE-FT) performs slightly better than the single-head version (FT-SH), suggesting that multi-head fine-tuning improves transferability and mitigates forgetting across diverse atomic configurations.

C. Molybdenum

In this example, we evaluate the capability of fine-tuned U-MLIPs to accurately describe mechanical properties associated with crystalline defects in metallic systems. Molybdenum (Mo), a body-centered-cubic (BCC) transition metal, is chosen as a representative case due to its well-characterized elastic properties and defect energetics, including vacancy formation and migration.[114]

Crystalline defects play a key role in determining the plasticity, strength, and diffusion behavior of metals. However, accurately capturing their highly localized and stress-sensitive nature remains a challenge for many interatomic potentials. This example is, therefore, designed to assess whether fine-tuning improves the quantitative prediction of defect-related energies and transition states in Mo. The atomic structure of bulk Mo is shown in Fig. 6. The data set used for fine-tuning is taken from Ref. 114.

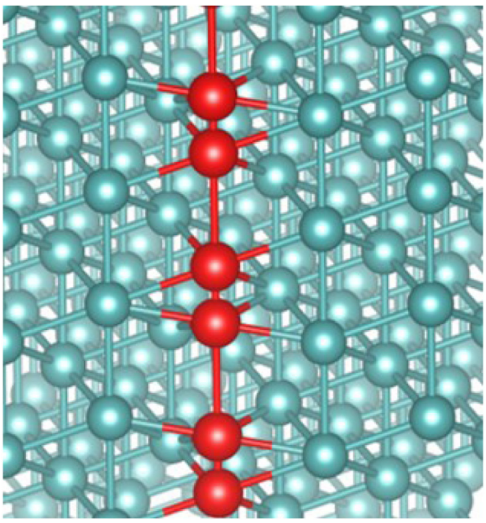


FIG. 6. Atomic structure of BCC Mo containing a dislocation and the associated local lattice distortion. The defect core is highlighted in red.

08 February 2026 12:36:01

© Author(s) 2026

TABLE VI. Elastic constants (C_{ij}), bulk modulus (B), and Poisson's ratio (ν) obtained from different interatomic potentials (EAM/FS, GAP, tabGAP, and NNIP), benchmarked against DFT results from this work. Values in parentheses indicate the percentage deviation from the DFT reference. Bold values highlight the best-performing results and those with relative errors below 3%.

	EAM[115]	GAP[116]	MACE-MP-0b3	Fine-tuning	DFT[114]
C_{11} (GPa)	**465 (1.31%)**	478 (4.74%)	251 (45.32%)	**452 (1.52%)**	459
C_{12} (GPa)	**161 (0.62%)**	**166 (2.47%)**	189 (16.67%)	174 (7.41%)	162
C_{44} (GPa)	109 (12.37%)	**108 (11.34%)**	47 (51.55%)	82 (15.46%)	97
B (GPa)	**263 (0.38%)**	270 (3.05%)	210 (19.85%)	**267 (1.91%)**	262
ν	0.26 (13.33%)	0.26 (13.33%)	0.43 (43.33%)	**0.28 (6.67%)**	0.30

Bulk validation. Table VI compares the elastic constants (C_{11}, C_{12}, and C_{44}), bulk modulus (B), and Poisson ratio (ν) of molybdenum predicted by different interatomic potentials (EAM, GAP, MACE-MP-0b3, and fine-tuned MACE-MP-0) against DFT and experimental reference values. The numbers in parentheses indicate the percentage error relative to experiment. The results show that fine-tuning the MACE-MP-0 model significantly improves its accuracy across all properties, reducing the error from 45.91% to 2.58% for C_{11}, from 56.88% to 14.77% for C_{44}, and from 48.27% to 3.45% for ν. While the original MACE-MP-0b3 underestimates the elastic constants and bulk modulus, fine-tuning narrows the gap to the DFT reference values, achieving accuracy comparable to EAM and GAP in most properties. We note that these results can be further improved by incorporating stress observables during training, which help the model better capture the elastic response of the material.

Generalized stacking fault energy. Figure 7 shows the generalized stacking fault energy (GSFE) profiles of Mo along the ⟨110⟩ [Fig. 7(a)] and ⟨112⟩ [Fig. 7(b)] directions. Table VII provides the corresponding GSFE barrier errors relative to DFT. The results compare the MACE foundation model, two fine-tuned models (FT-Default, which employs the default loss weights, and FT, which uses an increased force loss weight of 100.0), and the DFT reference data. The foundation model clearly underestimates the GSFE in both directions, leading to lower energy barriers. After fine-tuning, the predictions become much closer to the DFT values, especially near the peak and along the entire fault path. Between the two fine-tuned models, FT performs slightly better than FT-Default because increasing the force weighting helps the model learn local gradients of the potential energy surface more accurately. This is particularly relevant for GSFE, which depends on the shape of the energy landscape along the slip pathway and is highly sensitive to the predicted forces that govern atomic relaxation at each displacement.

D. Graphene–water interface

Graphene–water interfaces are prototypical solid–liquid systems with wide-ranging applications in sensing, electrochemistry, and nanofluidics. Capturing the interfacial structure and dynamics of such systems presents a significant challenge for interatomic potentials, especially in describing long-range interactions and subtle surface effects. This example illustrates the capability of fine-tuned U-MLIPs to model solid–liquid interfaces accurately, using a representative graphene–water system. The atomic structure

08 February 2026 12:36:01

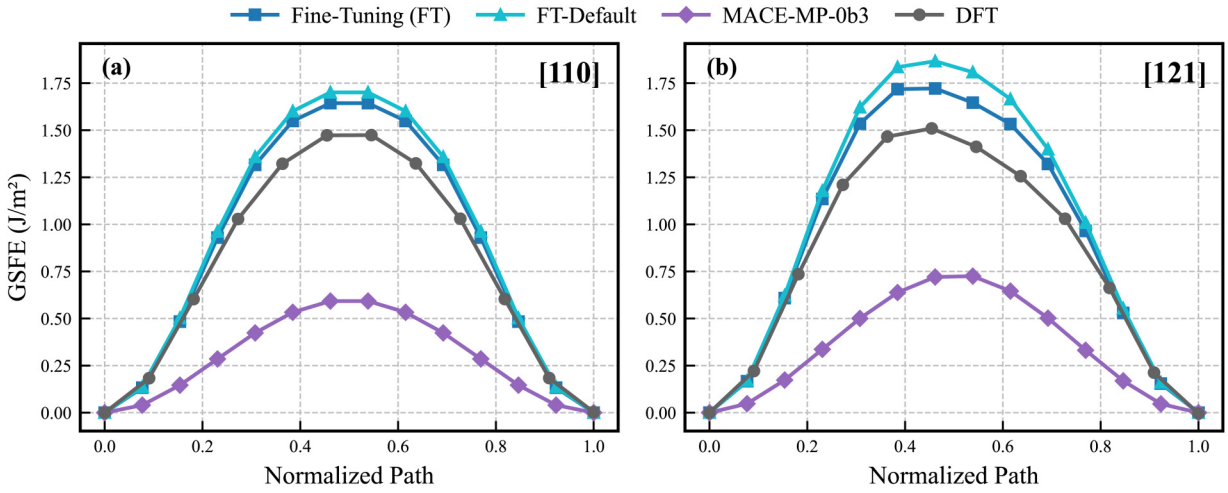


FIG. 7. GSFE profiles of Mo along the ⟨110⟩ (a) and ⟨112⟩ (b) directions. Two fine-tuned models are compared: FT-Default, which uses the default loss weights, and FT, which employs a force loss weight of 100.0.

© Author(s) 2026

TABLE VII. Generalized stacking fault energy (GSFE) barrier errors relative to DFT for the ⟨110⟩ and ⟨121⟩ slip directions. Results are reported for the MACE foundation model (MACE-MP-0b3) and two fine-tuned variants: FT-Default (using the default loss weights) and FT (using an increased force weight of 100.0). The FT model achieves the best agreement with DFT in both directions. Bold values indicate the best performance in each column.

Slip direction	MACE-MP-0b3	FT-Default	FT
⟨110⟩	0.88	0.23	**0.17**
⟨121⟩	0.78	0.36	**0.21**

of the graphene–water interface is illustrated in Fig. 8. The system contained over 372 atoms and was simulated for 200 ps with a 1 fs time step.

The graphene–water data set used to fine-tune the U-MLIP was generated by pre-equilibrating a periodic graphene–water interface with classical molecular dynamics (LAMMPS),[117,118] ensuring that no unphysical configurations from random generation methods (cf. Sec. II B) were included. Graphene was modeled with a Tersoff bond-order potential[119,120] and water with a rigid three-site model constrained by SHAKE; short-range interactions were described by Lennard-Jones terms,[121] and long-range electrostatics were described by Ewald summation under periodic boundary conditions. Canonical (NVT) sampling furnished equilibrated configurations, from which temporally uncorrelated snapshots were randomly selected for *ab initio* labeling. Single-point DFT labels were obtained in VASP[122] using the PAW[123] formalism and the PBE exchange–correlation functional[124] augmented with D3 dispersion[125] to capture graphene–water physisorption. The cutoff of 400 eV was chosen for the plane-wave basis, and electronic self-consistency was achieved when the total-energy change fell below 1×10^{-6} eV.

Figure 9 compares the molecular dynamics simulations of a graphene–water interface performed using LAMMPS-MACE with

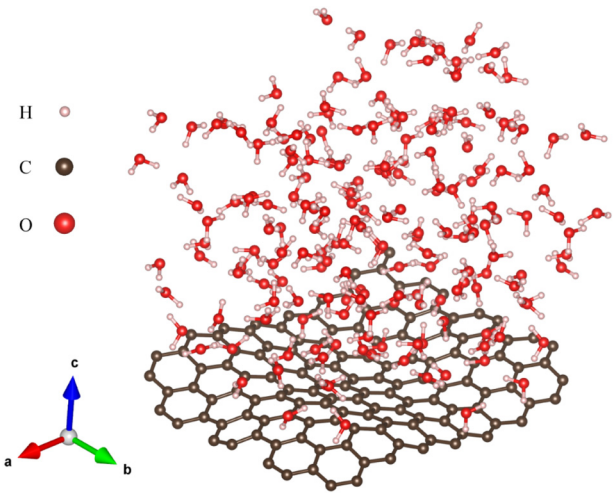


FIG. 8. Atomic structure of the graphene–water interface used in this study.

the fine-tuned model (MACE-FT-MD) and *ab initio* molecular dynamics (AIMD). Figure 9(a) shows the temperature evolution, with both simulations maintaining comparable temperature fluctuations around 300 K. Figure 9(b) presents the oxygen number-density profile along the z axis. The first peak corresponds to the graphene–water interfacial distance, dgw = 0.35 nm, which is in good agreement with the experimental value of 0.36 nm.[126] Due to the hydrophobic nature of graphene, a depletion layer forms at the interface, and dgw is defined to quantify this interfacial gap. This distance is a critical physical quantity for evaluating the accuracy of the interaction potentials and demonstrates the effectiveness of the fine-tuned model.

Table VIII reports the computational cost (wall-clock time in seconds) for 10 000-step molecular dynamics simulations performed with and without the cuEquivalence kernel, across systems of varying sizes (128–2900 atoms). For each configuration, simulations using cuEquivalence ("With") show consistently lower elapsed times compared to runs without it ("Without"), especially for larger systems. For instance, at 2900 atoms, cuEquivalence reduces the runtime from 5084 to 2619 s. The scaling factor improves from 0.76 (without cuEquivalence) to 0.41 (with cuEquivalence), indicating better parallel efficiency with the kernel enabled. These results suggest that the new kernel can serve as an efficient building block for large-scale and long-timescale molecular dynamics simulations.

E. Molecule/oxide interface

As a representative class of interfacial problems, molecule–oxide interactions are widely used to probe long-range phenomena such as charge transfer and polarization at insulating surfaces. In this work, we examine two systems. First, following Ref. 127, we consider a gold dimer (Au_2) adsorbed on Al-doped MgO(001), using the data set reported therein. Second, following Ref. 128, we study the anatase TiO_2 (101) surface in contact with NaCl–water electrolyte solutions across a range of pH values. Together, these cases provide stringent tests of whether fine-tuned foundation models can accurately capture interfacial physics that are often beyond the reach of conventional MLIPs.

Figure 10 compares the fine-tuned foundation model (MACE-FT) with the CACE-LR baseline reported in Ref. 127: (a) energy RMSE per atom (meV/atom) and (b) force RMSE (meV/Å). We do not report pre-trained model errors for the foundation checkpoint because differing DFT settings between our data set and the original training data can introduce misleading absolute offsets. In this RMSE comparison, MACE-FT attains accuracy comparable to, and in some cases slightly better than, CACE-LR, which employs a latent Ewald summation to represent long-range effects. These results indicate that fine-tuning can be competitive even without explicit long-range terms although such behavior should be assessed on a case-by-case basis. Evaluations on task-specific quantities of interest, such as adsorption energies, would require additional DFT calculations and are left for future work.

F. Solid–solid interface

Atomistic modeling of solid–solid interfaces is essential for understanding the synthesizability and stability of materials.[130]

08 February 2026 12:36:01

© Author(s) 2026

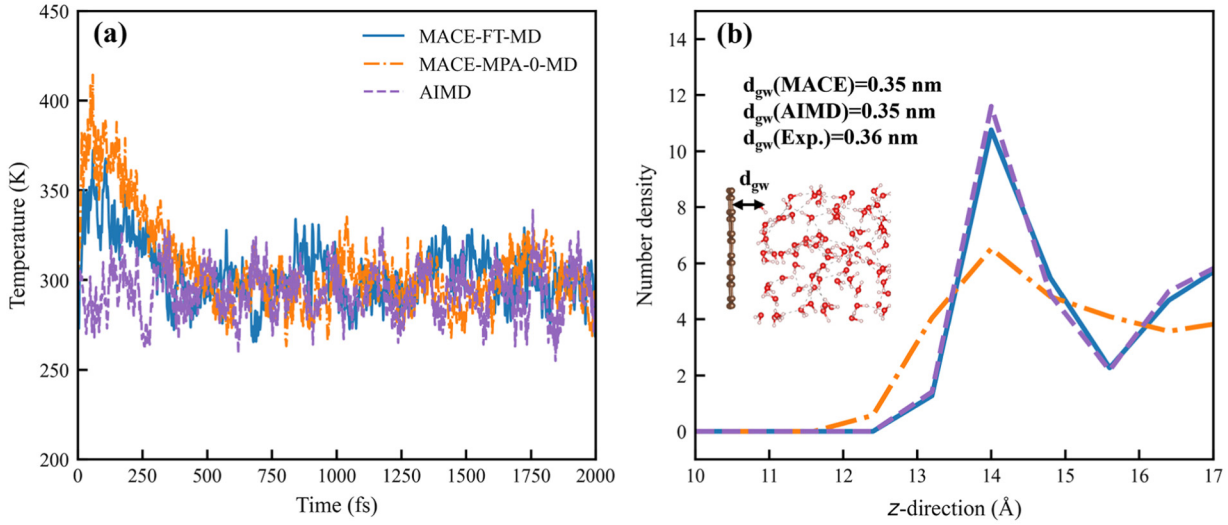


FIG. 9. Comparison of graphene–water interface simulations using MACE-FT-MD, MACE-MPA-0-MD, and AIMD. (a) Temperature evolution over time. (b) Oxygen number-density profile along the z axis. The first peak corresponds to the graphene–water distance, $d_{gw} = 0.35$ nm, which is in good agreement with the experimental value of 0.36 nm.[126] The inset illustrates the definition of d_{gw}. To make a comparison, we also plot the MD trajectory for the MACE-MPA-0 model without fine-tuning.

Such interfaces are notoriously challenging to model because their heterogeneous nature often requires long-period supercells and an accurate treatment of long-range interactions, including charge transfer and polarization, which are typically beyond the scope of conventional MLIPs. To assess whether fine-tuned foundation models can capture such effects, we consider two representative heterogeneous interfaces: LiCl(001)/GaF_3(001) from Ref. 127 and diamond/cubic boron nitride (C/cBN) from Ref. 131. The atomic structures of both systems are illustrated in Fig. 11.

Table IX summarizes the test RMSEs for energies and forces before and after fine-tuning along with reference results from Refs. 127 and 131. The CACE-LR method, which employs a latent Ewald summation to explicitly capture long-range interactions, already achieves strong accuracy for the LiCl/GaF_3 interface. Remarkably, the fine-tuned foundation model (MACE-FT) performs even better, despite lacking any explicit long-range correction. A similar trend is observed for the diamond/cubic boron nitride (cBN) interface, where MACE-FT outperforms both the original MACE-MPA-0 and the NEP models. We attribute this to the fact that large pre-trained models may implicitly encode aspects of long-range physics during pre-training, which fine-tuning then refines for the target interface. This observation suggests that foundation models, when fine-tuned, can generalize certain long-range effects beyond the reach of conventional MLIPs although such behavior is case by case and will be investigated further.

As summarized in Table X, the performance gain obtained from optimized hyperparameters is system-dependent. For the LiGePS data set, the optimized combination of `forces_weight` = 100, `lr` = 0.001, and `ema_decay` = 0.999 yields a reasonable improvement (about 10%) in both energy and force predictions. In contrast, for the LiCl/GaF_3 interface, the same hyperparameters bring only marginal gains. This observation highlights that the optimal configuration is case specific and depends on the physical characteristics and data distribution of each system. It further motivates the development of an automated system-adaptive workflow for hyperparameter optimization that will enable efficient and reproducible fine-tuning across diverse classes of materials. We plan to explore this direction in detail in future work.

TABLE VIII. Computational cost (wall-clock time in seconds) for 10 000-step molecular dynamics simulations with and without the use of the cuEquivalence kernel. The first column indicates the total number of atoms in the system, while the remaining columns report the elapsed time for each configuration. "With" denotes simulations utilizing the cuEquivalence kernel, and "Without" corresponds to runs without it. Bold values indicate the best performance in each column.

Atoms	128	372	1038	2900	Scaling
With (s)	710	800	1126	2619	**0.41**
Without (s)	658	1008	2031	5084	0.76

IV. SUMMARY AND PERSPECTIVE

This Tutorial provides a practical guide to fine-tuning universal machine-learned interatomic potentials (U-MLIPs), using the MACE-MP-0 model as a representative example. We cover the end-to-end process—from data set preparation and hyperparameter selection to model training and application, with the goal of helping researchers efficiently adapt pre-trained models to their systems of interest. The effectiveness of fine-tuning is demonstrated through representative examples, including solid-state electrolytes, stacking fault defects in metals, surface interactions in low-

08 February 2026 12:36:01

© Author(s) 2026

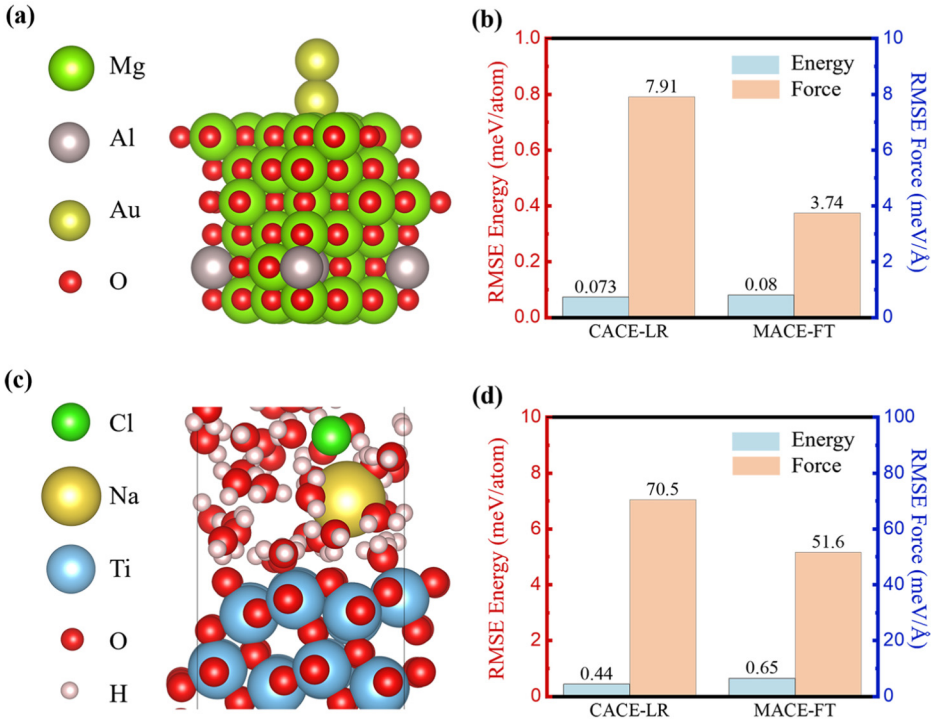


FIG. 10. Doped Au–MgO. (a) Atomic structure. (b) Dual-axis RMSE (energy, meV/atom; force, meV/Å). Oxide–electrolyte interface. (c) Atomic structure. (d) Dual-axis RMSE. Across both systems, MACE-FT matches CACE-LR while reducing force errors. CACE-LR values are taken from Ref. 127 for (a) and (b) and Ref. 129 for (c) and (d). MACE-FT values are from this work.

dimensional materials, and more complicated heterointerfaces. To support reproducibility, all code and data are made publicly available along with scripts for running molecular dynamics simulations using the RBMD platform.

While this Tutorial focuses on MACE-MP-0, the strategies discussed are readily transferable to other U-MLIP frameworks. We hope that this work will help lower the barrier to entry for applying fine-tuned MLIPs in atomistic simulations and encourage their broader use in both academic and industrial settings.

That said, caution is required when fine-tuning or applying MLIPs. These models approximate the potential energy surface purely from data and do not inherently incorporate physical laws or constraints. Their accuracy is ultimately determined by the coverage and quality of the training data set. We point out several important considerations and open directions for future development of MLIPs.

A. Uncertainty quantification and active fine-tuning

UQ is essential for evaluating model reliability and guiding data selection in active learning and fine-tuning. Techniques such as ensemble variance, dropout inference, and Bayesian approximations help identify regions of high predictive uncertainty, enabling targeted sampling and noise reduction. In fine-tuning, UQ reduces data redundancy and ensures that added configurations meaningfully enhance model performance. A central challenge is the automated construction of fine-tuning datasets informed by uncertainty, task relevance, or physical constraints. Once such data sets can be systematically generated, selecting or adapting models becomes significantly more efficient. Developing robust pipelines for uncertainty-aware data generation is, thus, key to scalable, task-adaptive fine-tuning.[132]

B. Building intermediate-sized MLIPs for specific tasks

Although U-MLIPs demonstrate excellent generalization across a wide range of material systems, they may not be optimal for small- to medium-sized systems or domain-specific tasks. In such cases, it may be more effective to either fine-tune a compact pre-trained model or train a lightweight architecture from scratch on a focused data set.[133,134] This approach offers a favorable trade-off between interpretability, computational efficiency, and predictive accuracy, particularly when computational resources are limited or when real-time inference is required for large-scale simulations. Lightweight variants of ACE and other descriptor-based models remain valuable tools for such targeted applications.

C. The need for comprehensive benchmarking platforms for MLIPs

As MLIPs continue to evolve into foundation models with broad applicability, there is an increasing need for benchmarking platforms that go beyond traditional accuracy metrics such as RMSE. Evaluating an MLIP's true usefulness requires not only numerical accuracy but also physical fidelity, generalization ability, and reliability in practical simulations. Conventional benchmarks often fail to capture these dimensions, making it difficult to assess whether a model is suitable for deployment in real-world applications. To address this gap, recent efforts have begun to develop

08 February 2026 12:36:01

© Author(s) 2026

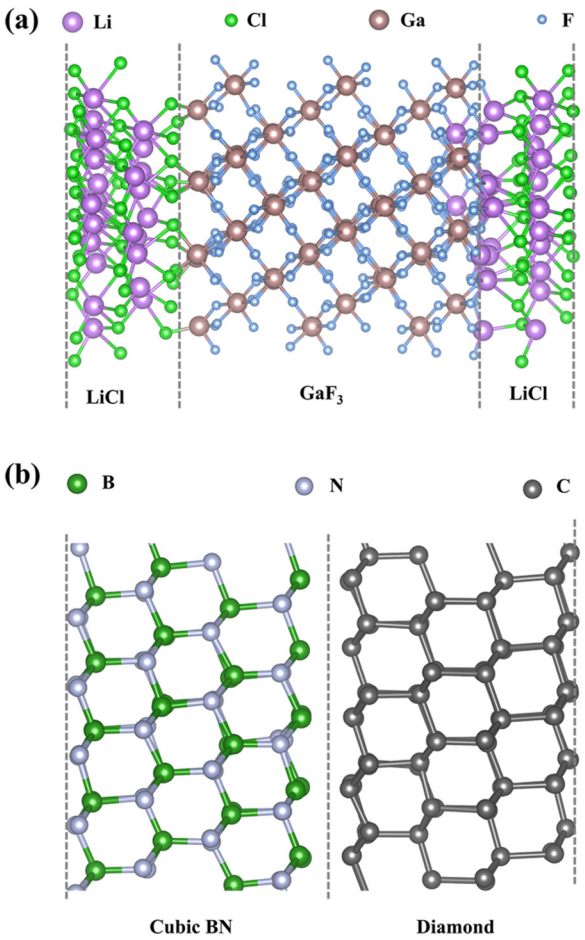


FIG. 11. Atomic structures used in this study: (a) LiCl(001)/GaF_3(001) interface and (b) diamond/cubic boron nitride (C/cBN) interface.

more comprehensive evaluation frameworks that emphasize model robustness, transferability, and physics-informed behavior. One notable example is *MLIP Arena*,[135] a newly proposed platform that offers architecture-agnostic, task-oriented benchmarking of open-source, open-weight MLIPs. It represents a promising step toward standardized, holistic assessment of model performance in complex material modeling tasks.

TABLE IX. Test RMSEs for energies (E, meV/atom) and forces (F, meV/Å) for the LiCl (001)/GaF_3(001) and C/cBN interfaces. Values for CACE-LR and NEP are taken from Refs. 127 and 131. The fine-tuned MACE model (FT) consistently achieves the best accuracy across all cases. Bold values indicate the best performance in each column.

System	Metric	MACE-MPA-0	FT	Baseline
LiCl/GaF_3	E	97.04	**0.09**	—
	F	179.2	**18.23**	67.8
C/cBN	E	48.09	**3.72**	7.00
	F	171.3	**18.78**	135.3

TABLE X. Comparison between default and optimized hyperparameter settings for two representative systems. Reported values include test RMSEs for energies (E, meV/atom) and forces (F, meV/Å), as well as the relative force error (Rel. F, %). The FT-Default model employs the standard MACE hyperparameters, whereas FT adopts the optimized values identified in Table V. Δ denotes the relative improvement from the default to the optimized configuration.

System	Metric	FT-Default	FT	Δ (%)
LiGePS	E	0.28	0.25	10.7
	F	15.76	14.09	10.6
	Rel. F	1.87	1.67	10.7
LiCl/GaF_3	E	0.09	0.09	0.0
	F	18.28	17.55	4.0
	Rel. F	3.81	3.66	3.9

D. Post-training strategies

After pre-training a foundation model, post-training serves as a crucial step to further adapt and optimize the model for specific downstream applications. One useful post-training approach is *model distillation*,[136,137] where a smaller model learns to reproduce the predictions of a larger, fine-tuned foundation model. This technique can generate compact task-specific potentials that run much faster in MD simulations while maintaining comparable accuracy for the target system. We briefly mention this here as a possible next step for extending fine-tuned foundation models, with planned implementation and benchmarking on the RBMD platform.

ACKNOWLEDGMENTS

Z. Xu and T. Zhao acknowledge the support from the National Natural Science Foundation of China (NNSFC) (Grant Nos. 12426304 and 12325113) and the Science and Technology Commission of Shanghai Municipality (Grant No. 23JC1402300). T. Zhao acknowledges the support from NNSFC (Grant No. 22508245) and the Natural Science Foundation of Chongqing (Grant No. CSTB2024NSCQ-MSX1238). This research used supercomputing resources of the National Supercomputing Center (KISTI) with technical support (KSC-2024-CRE-0513). We thank the original developers of MACE, particularly Professor Gábor Csányi's group, for their insightful discussions and valuable contributions. We also appreciate the helpful suggestions and exchanges on the MACE GitHub repository (https://github.com/ACEsuit/mace) that have informed and improved this tutorial on fine-tuning MACE foundation models. Readers are encouraged to consult the official MACE documentation (https://mace-docs.readthedocs.io/en/latest/), upon which parts of this tutorial are based. Finally, we thank the author of the GitHub repository (https://github.com/BingqingCheng/cace-lr-fit) for generously sharing the data set that enabled our last two numerical examples.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

08 February 2026 12:36:01

© Author(s) 2026

Author Contributions

Xiaoqing Liu: Data curation (equal); Formal analysis (equal); Investigation (equal); Methodology (equal); Software (equal); Writing – original draft (equal). **Kehan Zeng:** Data curation (equal); Investigation (equal); Software (equal). **Zedong Luo:** Data curation (equal); Software (equal). **Yangshuai Wang:** Conceptualization (lead); Investigation (equal); Supervision (equal); Writing – review & editing (equal). **Teng Zhao:** Conceptualization (equal); Formal analysis (equal); Funding acquisition (equal); Project administration (equal); Validation (equal); Writing – review & editing (equal). **Zhenli Xu:** Funding acquisition (equal); Supervision (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The datasets and source code supporting all numerical results presented in this work are publicly available in the Github tutorial repository at https://github.com/John2021-hub/mace-ft-tutorial.git, Ref. 141.

APPENDIX: MACE ARCHITECTURE

In this subsection, we introduce the main idea behind the construction of the MACE architecture. We provide only a brief overview here and refer interested readers to Ref. 6 for more detailed constructional insights.

First, a graph is defined by connecting two nodes (atoms) with an edge if they are within each other's local environment. The local environment, $\mathcal{N}(i)$, consists of all atoms j surrounding a central atom i such that $\|\mathbf{r}_{ij}\| \leq r_{\text{cut}}$, where $\mathbf{r}_{ij}$ is the vector from atom i to atom j and r_{cut} is a cutoff hyperparameter. The feature vector of node i is denoted by $\mathbf{h}_i^{(t)}$, expressed in the spherical harmonic basis, with indices l and m. The superscript t represents the iteration step (analogous to the "layers" in graph neural networks' message passing). These node features, $\mathbf{h}_i^{(t)}$, reflect the chemical environment of the atoms.

It is worth noting that all MACE models consist of only two layers. Due to the specific construction, MACE can be regarded as a higher-order message-passing neural network. This enables it to achieve comparable expressiveness with just two layers, which is a key strength of the MACE model.

The node features on the zeroth layer denoted as $\mathbf{h}_i^{(0)}$ are initialized as a (learnable) embedding of the chemical elements with atomic numbers z_i into k channels,

$$h_{i,k00}^{(0)} = \sum_z W_{kz}\delta_{zz_i}. \tag{A1}$$

This type of mapping has been widely applied to graph neural networks[7,138] and other models.[44,139]

Next, for each atom, the features of its neighbors are combined with the interatomic displacement vectors, $\mathbf{r}_{ij}$, to form the one-particle basis $\phi_{ij,knl_1l_2m_2}^{(t)}$. The construction of MACE employs the idea from the ACE architecture, which is a natural many-body representation for capturing the symmetry of many-body particle systems. The radial distance r_{ij} is used as an input into a learnable radial function $R(r_{ij})$, with several outputs that correspond to different ways in which the displacement vector and node features can be combined while maintaining equivariance,[140]

$$\phi_{ij,knl_1l_2m_3}^{(t)} = \sum_{l_1m_1l_2m_2} C_{\eta_1,l_1m_1,l_2m_2}^{l_3m_3} R_{knl_1l_2l_3}^{(t)}(r_{ij}) Y_{l_1m_1}^{m_1}(\hat{r}_{ij}) h_{j,kl_2m_2}^{(t)}, \tag{A2}$$

where Y_l^m are the spherical harmonics and $C_{\eta_1,l_1m_1,l_2m_2}^{l_3m_3}$ denotes the Clebsch–Gordan coefficients. There are multiple ways of constructing an equivariant combination with a given symmetry (l_3, m_3), and these multiplicities are enumerated by the path index η_1.[43]

The one-particle basis ϕ is summed over the neighborhood and expanded as a linear combination of k channels with learnable weights to form the permutation-invariant atomic basis A_i,

$$A_{i,kl3m3}^{(t)} = \sum_{\tilde{k},\eta_1} W_{\tilde{k}\tilde{k}kn_1l_3} \sum_{j\in\mathcal{N}(i)} \phi_{ij,\tilde{k}\eta_1l_3m_3}^{(t)}. \tag{A3}$$

Higher-order (many-body) symmetric features are created on each atom by taking products of the atomic basis, A, with itself ν times, resulting in the "product basis." The product basis is then contracted with the generalized Clebsch–Gordan coefficients $C_{\eta_\nu lm}^{LM}$ to obtain the equivariant higher-order basis, B_i,[43]

$$B_{i,\eta_\nu kLM}^{(t),\nu} = \sum_{lm} C_{\eta_\nu lm}^{LM} \prod_{\xi=1}^{\nu} A_{i,kl_\xi m_\xi}^{(t)}, \tag{A4}$$

where bold lm denotes the ν-tuple of l and m values and similarly to Eq. (A2), η_ν enumerates the number of possible couplings to create the features with equivariance LM.

Finally, a "message" m_i is created on each atom as a learnable linear combination of the equivariant many-body features,

$$m_{i,kLM}^{(t)} = \sum_{\nu}\sum_{\eta_\nu} W_{zi\eta_\nu kLM}^{(t),\nu} B_{i,\eta_\nu kLM}^{(t),\nu}. \tag{A5}$$

The recursive update of the node features $(t:0\rightarrow 2)$ is obtained by adding the message to the atoms' features from the previous iteration, with weights that depend on the chemical element (z_i) that are also responsible for the mixing of the chemical embedding k channels,

$$h_{i,kLM}^{(t+1)} = \sum_{\tilde{k}} W_{kL,\tilde{k}}^{(t)} m_{i,\tilde{k}LM}^{(t)} + \sum_{\tilde{k}} W_{kziL,\tilde{k}}^{(t)} h_{i,\tilde{k}LM}^{(t)}. \tag{A6}$$

The site energy is a sum of readout functions applied to node features from the first and second layers. The readout function is defined as a linear combination of rotationally invariant node features for the first layer, and as a multi-layer perceptron (MLP) for the second layer,

$$E_i = \sum_{t=1}^{2} E_i^{(t)} = \sum_{t=1}^{2} \mathcal{R}^{(t)}\left(h_i^{(t)}\right), \tag{A7}$$

where

$$\mathcal{R}^{(t)}\left(h_i^{(t)}\right) = \begin{cases} \sum_k W_k^{(t)} h_{i,k00}^{(t)}, & \text{for } t=1, \\ \text{MLP}\left(\left\{h_{i,k00}^{(t)}\right\}_k\right), & \text{for } t=2. \end{cases} \tag{A8}$$

The forces and stresses on the atoms are calculated by taking analytical derivatives of the total potential energy with respect to the positions of the atoms.

08 February 2026 12:36:01

© Author(s) 2026

REFERENCES

[1]J. Behler and M. Parrinello, "Generalized neural-network representation of high-dimensional potential-energy surfaces," Phys. Rev. Lett. **98**, 146401 (2007).
[2]A. P. Bartók, M. C. Payne, R. Kondor, and G. Csányi, "Gaussian approximation potentials: The accuracy of quantum mechanics, without the electrons," Phys. Rev. Lett. **104**, 136403 (2010).
[3]A. V. Shapeev, "Moment tensor potentials: A class of systematically improvable interatomic potentials," Multiscale Model. Simul. **14**, 1153 (2016).
[4]H. Wang, L. Zhang, J. Han *et al.*, "DeePMD-kit: A deep learning package for many-body potential energy representation and molecular dynamics," Comput. Phys. Commun. **228**, 178 (2018).
[5]R. Drautz, "Atomic cluster expansion for accurate and transferable interatomic potentials," Phys. Rev. B **99**, 014104 (2019).
[6]I. Batatia, D. P. Kovacs, G. Simm, C. Ortner, and G. Csányi, "MACE: Higher order equivariant message passing neural networks for fast and accurate force fields," Adv. Neural Inf. Process. Syst. **35**, 11423 (2022).
[7]K. Schütt, P.-J. Kindermans, H. E. Sauceda Felix, S. Chmiela, A. Tkatchenko, and K.-R. Muller, "SchNet: A continuous-filter convolutional neural network for modeling quantum interactions," in *31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA* (NeurIPS, 2017), available at https://proceedings.neurips.cc/paper_files/paper/2017/file/303ed4c69846ab36c2904d3ba8573050-Paper.pdf.
[8]S. Batzner, A. Musaelian, L. Sun, M. Geiger, J. P. Mailoa, M. Kornbluth, N. Molinari, T. E. Smidt, and B. Kozinsky, "E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials," Nat. Commun. **13**, 2453 (2022).
[9]B. Cheng, "Cartesian atomic cluster expansion for machine learning interatomic potentials," npj Comput. Mater. **10**, 157 (2024).
[10]A. Musaelian, S. Batzner, A. Johansson, L. Sun, C. J. Owen, M. Kornbluth, and B. Kozinsky, "Learning local equivariant representations for large-scale atomistic dynamics," Nat. Commun. **14**, 579 (2023).
[11]J. S. Smith, O. Isayev, and A. E. Roitberg, "ANI-1: An extensible neural network potential with DFT accuracy at force field computational cost," Chem. Sci. **8**, 3192 (2017).
[12]A. Bochkarev, Y. Lysogorskiy, and R. Drautz, "Graph atomic cluster expansion for semilocal interactions beyond equivariant message passing," Phys. Rev. X **14**, 021036 (2024).
[13]A. P. Thompson, L. P. Swiler, C. R. Trott, S. M. Foiles, and G. J. Tucker, "Spectral neighbor analysis method for automated generation of quantum-accurate interatomic potentials," J. Comput. Phys. **285**, 316 (2015).
[14]S. R. Xie, M. Rupp, and R. G. Hennig, "Ultra-fast interpretable machine-learning potentials," npj Comput. Mater. **9**, 162 (2023).
[15]A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus *et al.*, "The atomic simulation environment—A Python library for working with atoms," J. Phys.: Condens. Matter **29**, 273002 (2017).
[16]A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen *et al.*, "LAMMPS—A flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales," Comput. Phys. Commun. **271**, 108171 (2022).
[17]O. T. Unke, S. Chmiela, H. E. Sauceda, M. Gastegger, I. Poltavsky, K. T. Schutt, A. Tkatchenko, and K.-R. Müller, "Machine learning force fields," Chem. Rev. **121**, 10142 (2021).
[18]V. Botu, R. Batra, J. Chapman, and R. Ramprasad, "Machine learning force fields: Construction, validation, and outlook," J. Phys. Chem. C **121**, 511 (2017).
[19]R. Jacobs, D. Morgan, S. Attarian, J. Meng, C. Shen, Z. Wu, C. Y. Xie, J. H. Yang, N. Artrith, B. Blaiszik *et al.*, "A practical guide to machine learning interatomic potentials–status and future," Curr. Opin. Solid State Mater. Sci. **35**, 101214 (2025).
[20]F. Musil, A. Grisafi, A. P. Bartók, C. Ortner, G. Csányi, and M. Ceriotti, "Physics-inspired structural representations for molecules and materials," Chem. Rev. **121**, 9759 (2021).
[21]I. Poltavsky and A. Tkatchenko, "Machine learning force fields: Recent advances and remaining challenges," J. Phys. Chem. Lett. **12**, 6551 (2021).
[22]I. Batatia, P. Benner, Y. Chiang, A. M. Elena, D. P. Kovács, J. Riebesell, X. R. Advincula, M. Asta, W. J. Baldwin, N. Bernstein *et al.*, "A foundation model for atomistic materials chemistry," arXiv:2401.00096 (2023).
[23]B. Deng, P. Zhong, K. Jun, J. Riebesell, K. Han, C. J. Bartel, and G. Ceder, "CHGNet as a pretrained universal neural network potential for charge-informed atomistic modelling," Nat. Mach. Intell. **5**, 1031 (2023).
[24]A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk, "Scaling deep learning for materials discovery," Nature **624**, 80 (2023).
[25]D. Zhang, X. Liu, X. Zhang, C. Zhang, C. Cai, H. Bi, Y. Du, X. Qin, A. Peng, J. Huang *et al.*, "DPA-2: A large atomic model as a multi-task learner," npj Comput. Mater. **10**, 293 (2024).
[26]K. Choudhary, B. DeCost, L. Major, K. Butler, J. Thiyagalingam, and F. Tavazza, "Unified graph neural network force-field for the periodic table: Solid state applications," Digital Discovery **2**, 346 (2023).
[27]C. Chen and S. P. Ong, "A universal graph deep learning interatomic potential for the periodic table," Nat. Comput. Sci. **2**, 718 (2022).
[28]M. Awais, M. Naseer, S. Khan, R. M. Anwer, H. Cholakkal, M. Shah, M.-H. Yang, and F. S. Khan, "Foundation models defining a new era in vision: A survey and outlook," IEEE Trans. Pattern Anal. Mach. Intell. **47**, 2245 (2025).
[29]L. Chanussot, A. Das, S. Goyal, T. Lavril, M. Shuaibi, M. Riviere, K. Tran, J. Heras-Domingo, C. Ho, W. Hu *et al.*, "Open catalyst 2020 (OC20) dataset and community challenges," ACS Catal. **11**, 6059 (2021).
[30]J. M. Bowman, C. Qu, R. Conte, A. Nandi, P. L. Houston, and Q. Yu, "The MD17 datasets from the perspective of datasets for gas-phase "small" molecule potentials," J. Chem. Phys. **156**, 240901 (2022).
[31]L. Barroso-Luque, M. Shuaibi, X. Fu, B. M. Wood, M. Dzamba, M. Gao, A. Rizvi, C. L. Zitnick, and Z. W. Ulissi, "Open Materials 2024 (OMat24) inorganic materials dataset and models," arXiv:2410.12771 (2024).
[32]K. T. Schütt, H. E. Sauceda, P.-J. Kindermans, A. Tkatchenko, and K.-R. Müller, "SchNet—A deep learning architecture for molecules and materials," J. Chem. Phys. **148**, 241722 (2018).
[33]S. Takamoto, D. Okanohara, Q.-J. Li, and J. Li, "Towards universal neural network interatomic potential," J. Materiomics **9**, 447 (2023).
[34]D. Zhang, H. Bi, F.-Z. Dai, W. Jiang, L. Zhang, and H. Wang, "DPA-1: Pretraining of attention-based deep potential model for molecular simulation," arXiv:2208.08236 (2022).
[35]D. Zhang, A. Peng, C. Cai, W. Li, Y. Zhou, J. Zeng, M. Guo, C. Zhang, B. Li, H. Jiang *et al.*, "Graph neural network model for the era of large atomistic models," arXiv:2506.01686 (2025).
[36]Y. Park, J. Kim, S. Hwang, and S. Han, "Scalable parallel algorithm for graph neural network interatomic potentials in molecular dynamics simulations," J. Chem. Theory Comput. **20**, 4857 (2024).
[37]H. Yang, C. Hu, Y. Zhou, X. Liu, Y. Shi, J. Li, G. Li, Z. Chen, S. Chen, C. Zeni *et al.*, "MatterSim: A deep learning atomistic model across elements, temperatures and pressures," arXiv:2405.04967 (2024).
[38]Y.-L. Liao, B. Wood, A. Das, and T. Smidt, "EquiformerV2: Improved equivariant transformer for scaling to higher-degree representations," in *The Twelfth International Conference on Learning Representations, Vienna, Austria* (ICLR, 2024), available at https://openreview.net/forum?id=mCOBKZmrzD.
[39]T. Liang, K. Xu, E. Lindgren, Z. Chen, R. Zhao, J. Liu, E. Berger, B. Tang, B. Zhang, Y. Wang *et al.*, "NEP89: Universal neuroevolution potential for inorganic and organic materials across 89 elements," arXiv:2504.21286 (2025).
[40]Z. Fan, Z. Zeng, C. Zhang, Y. Wang, K. Song, H. Dong, Y. Chen, and T. Ala-Nissila, "Neuroevolution machine learning potentials: Combining high accuracy and low cost in atomistic simulations and application to heat transport," Phys. Rev. B **104**, 104309 (2021).
[41]B. M. Wood, M. Dzamba, X. Fu, M. Gao, M. Shuaibi, L. Barroso-Luque, K. Abdelmaqsoud, V. Gharakhanyan, J. R. Kitchin, D. S. Levine *et al.*, "UMA: A family of universal models for atoms," arXiv:2506.23971 (2025).

08 February 2026 12:36:01

© Author(s) 2026

[42]X. Fu, B. M. Wood, L. Barroso-Luque, D. S. Levine, M. Gao, M. Dzamba, and C. L. Zitnick, "Learning smooth and expressive interatomic potentials for physical property prediction," arXiv:2502.12147 (2025).
[43]G. Dusson, M. Bachmayr, G. Csányi, R. Drautz, S. Etter, C. van der Oord, and C. Ortner, "Atomic cluster expansion: Completeness, efficiency and stability," J. Comput. Phys. **454**, 110946 (2022).
[44]J. P. Darby, D. P. Kovács, I. Batatia, M. A. Caro, G. L. Hart, C. Ortner, and G. Csányi, "Tensor-reduced atomic density representations," Phys. Rev. Lett. **131**, 028001 (2023).
[45]Z. Li, T. Wen, Y. Zhang, X. Liu, C. Zhang, A. Pattamatta, X. Gong, B. Ye, H. Wang, L. Zhang *et al.*, "An extendable cloud-native alloy property explorer," arXiv:2404.17330 (2024).
[46]B. Focassio, L. P. M. Freitas, and G. R. Schleder, "Performance assessment of universal machine learning interatomic potentials: Challenges and directions for materials' surfaces," ACS Appl. Mater. Interfaces **17**, 13111 (2025).
[47]B. Deng, Y. Choi, P. Zhong, J. Riebesell, S. Anand, Z. Li, K. Jun, K. A. Persson, and G. Ceder, "Systematic softening in universal machine learning interatomic potentials," npj Comput. Mater. **11**, 1 (2025).
[48]H. Yu, M. Giantomassi, G. Materzanini, J. Wang, and G.-M. Rignanese, "Systematic assessment of various universal machine-learning interatomic potentials," Mater. Genome Eng. Adv. **2**, e58 (2024).
[49]E. O. Pyzer-Knapp, M. Manica, P. Staar, L. Morin, P. Ruch, T. Laino, J. R. Smith, and A. Curioni, "Foundation models for materials discovery–current state and future directions," npj Comput. Mater. **11**, 61 (2025).
[50]F. Shuang, Z. Wei, K. Liu, W. Gao, and P. Dey, "Universal machine learning interatomic potentials poised to supplant DFT in modeling general defects in metals and random alloys," arXiv:2502.03578 (2025).
[51]H. Du, J. Hui, L. Zhang, and H. Wang, "Universal machine learning interatomic potentials are ready for solid ion conductors," arXiv:2502.09970 (2025).
[52]H. Lee, V. I. Hegde, C. Wolverton, and Y. Xia, "Accelerating high-throughput phonon calculations via machine learning universal potentials," Mater. Today Phys. **53**, 101688 (2025).
[53]S. P. Niblett, P. Kourtis, I.-B. Magdău, C. P. Grey, and G. Csányi, "Transferability of datasets between machine-learning interaction potentials," arXiv:2409.05590 (2024).
[54]L. Casillas-Trujillo, A. S. Parackal, R. Armiento, and B. Alling, "Evaluating and improving the predictive accuracy of mixing enthalpies and volumes in disordered alloys from universal pretrained machine learning potentials," Phys. Rev. Mater. **8**, 113803 (2024).
[55]See https://colab.research.google.com/drive/1oCSVfMhWrqHTeHbKgUSQN9hTKxLzoNyb for "Mace fine-tuning tutorial (colab notebook)" (accessed October 18, 2025).
[56]W. Gao, T. Zhao, Y. Guo, J. Liang, H. Liu, M. Luo, Z. Luo, W. Qin, Y. Wang, Q. Zhou *et al.*, "RBMD: A molecular dynamics package enabling to simulate 10 million all-atom particles in a single graphics processing unit," Commun. Comput. Phys. **39**, 296 (2025).
[57]S. Jin, L. Li, and J.-G. Liu, "Random batch methods (RBM) for interacting particle systems," J. Comput. Phys. **400**, 108877 (2020).
[58]S. Jin, L. Li, Z. Xu, and Y. Zhao, "A random batch Ewald method for particle systems with Coulomb interactions," SIAM J. Sci. Comput. **43**, B937 (2021).
[59]J. Liang, P. Tan, Y. Zhao, L. Li, S. Jin, L. Hong, and Z. Xu, "Superscalability of the random batch Ewald method," J. Chem. Phys. **156**, 014114 (2022).
[60]J. Liang, Z. Xu, and Y. Zhao, "Improved random batch Ewald method in molecular dynamics simulations," J. Phys. Chem. A **126**, 3583 (2022).
[61]W. C. Witt, C. van der Oord, E. Gelžinytė, T. Järvinen, A. Ross, J. P. Darby, C. H. Ho, W. J. Baldwin, M. Sachs, J. Kermode *et al.*, "ACEpotentials.jl: A Julia implementation of the atomic cluster expansion," J. Chem. Phys. **159**, 164101 (2023).
[62]L. Zhang, B. Onat, G. Dusson, A. McSloy, G. Anand, R. J. Maurer, C. Ortner, and J. R. Kermode, "Equivariant analytical mapping of first principles Hamiltonians to accurate and transferable materials models," npj Comput. Mater. **8**, 158 (2022).
[63]Y. Wang, S. Patel, and C. Ortner, "A theoretical case study of the generalization of machine-learned potentials," Comput. Methods Appl. Mech. Eng. **422**, 116831 (2024).
[64]T. Torabi, C. Ortner, and Y. Wang, "Surrogate models for vibrational entropy based on a spatial decomposition," Multiscale Model. Simul. **23**, 514 (2025).
[65]Y. Wang, G. Csanyi, and C. Ortner, "Many-body coarse-grained molecular dynamics with the atomic cluster expansion," arXiv:2502.04661 (2025).
[66]S. Luo, T. Chen, and A. S. Krishnapriyan, "Enabling efficient equivariant operations in the Fourier basis via Gaunt tensor products," arXiv:2401.10216 (2024).
[67]A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder *et al.*, "Commentary: The materials project: A materials genome approach to accelerating materials innovation," APL Mater. **1**, 011002 (2013).
[68]N. Tajbakhsh, J. Y. Shin, S. R. Gurudu, R. T. Hurst, C. B. Kendall, M. B. Gotway, and J. Liang, "Convolutional neural networks for medical image analysis: Full training or fine tuning?," IEEE Trans. Med. Imaging **35**, 1299 (2016).
[69]M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," Science **349**, 255 (2015).
[70]H. Zhang, G. Li, J. Li, Z. Zhang, Y. Zhu, and Z. Jin, "Fine-tuning pre-trained language models effectively by optimizing subnetworks adaptively," Adv. Neural Inf. Process. Syst. **35**, 21442 (2022).
[71]J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Association for Computational Linguistics, 2019), pp. 4171–4186.
[72]J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, "Biobert: A pre-trained biomedical language model for biomedical text mining," Bioinformatics **36**, 1234 (2020).
[73]S. Kim, H. Yang, Y. Kim, Y. Hong, and E. Park, "Hydra: Multi-head low-rank adaptation for parameter efficient fine-tuning," Neural Networks **178**, 106414 (2024).
[74]M. Radova, W. G. Stark, C. S. Allen, R. J. Maurer, and A. P. Bartók, "Fine-tuning foundation models of materials interatomic potentials with frozen transfer learning," npj Comput. Mater. **11**, 237 (2025).
[75]H. Kaur, F. Della Pia, I. Batatia, X. R. Advincula, B. X. Shi, J. Lan, G. Csányi, A. Michaelides, and V. Kapil, "Data-efficient fine-tuning of foundational models for first-principles quality sublimation enthalpies," Faraday Discuss. **256**, 120 (2025).
[76]C. Wang, S. Hu, G. Tan, and W. Jia, "ELoRA: Low-rank adaptation for equivariant GNNs," in *Forty-Second International Conference on Machine Learning, Vancouver, Canada* (PMLR, 2025).
[77]J. Moellmann and S. Grimme, "DFT-D3 study of some molecular crystals," J. Phys. Chem. C **118**, 7615 (2014).
[78]P. Novelli, G. Meanti, P. J. Buigues, L. Rosasco, M. Parrinello, M. Pontil, and L. Bonati, "Fast and Fourier features for transfer learning of interatomic potentials," npj Comp. Mater. **11**(1), 293 (2025).
[79]M. Kulichenko, B. Nebgen, N. Lubbers, J. S. Smith, K. Barros, A. E. Allen, A. Habib, E. Shinkle, N. Fedik, Y. W. Li *et al.*, "Data generation for machine learning interatomic potentials and beyond," Chem. Rev. **124**, 13681 (2024).
[80]"AIS-Square datasets and models," AIS-Square Platform, https://www.aissquare.com (2025).
[81]Z. Fan, "NEP dataset repository," GitLab, https://gitlab.com/brucefan1983/nep-data (2025).
[82]P. Ying, C. Qian, R. Zhao, Y. Wang, F. Ding, S. Chen, and Z. Fan, "Advances in modeling complex materials: The rise of neuroevolution potentials," arXiv:2501.11191 (2025).
[83]F. Eriksson, E. Fransson, and P. Erhart, "The hiPhive package for the extraction of high-order force constants by machine learning," Adv. Theory Simul. **2**, 1800184 (2019).
[84]C. van der Oord, M. Sachs, D. P. Kovács, C. Ortner, and G. Csányi, "Hyperactive learning for data-driven interatomic potentials," npj Comput. Mater. **9**, 168 (2023).

08 February 2026 12:36:01

© Author(s) 2026

[85]D. Marx and J. Hutter, *Ab Initio Molecular Dynamics: Basic Theory and Advanced Methods* (Cambridge University Press, 2009).
[86]G. Kresse and J. Hafner, "*Ab initio* molecular dynamics for liquid metals," Phys. Rev. B **47**, 558 (1993).
[87]R. C. Bernardi, M. C. R. Melo, and K. Schulten, "Enhanced sampling techniques in molecular dynamics simulations of biological systems," Biochim. Biophys. Acta, Gen. Subj. **1850**, 872 (2015).
[88]Y. I. Yang, Q. Shao, J. Zhang, L. Yang, and Y. Q. Gao, "Enhanced sampling in molecular dynamics," J. Chem. Phys. **151**, 070902 (2019).
[89]A. Zhu, S. Batzner, A. Musaelian, and B. Kozinsky, "Fast uncertainty estimates in deep learning interatomic potentials," J. Chem. Phys. **158**, 164111 (2023).
[90]M. Kellner and M. Ceriotti, "Uncertainty quantification by direct propagation of shallow ensembles," Mach. Learn. Sci. Technol. **5**, 035006 (2024).
[91]W. Edeling, M. Vassaux, Y. Yang, S. Wan, S. Guillas, and P. V. Coveney, "Global ranking of the sensitivity of interaction potential contributions within classical molecular dynamics force fields," npj Comput. Mater. **10**, 87 (2024).
[92]S. Venturi, R. L. Jaffe, and M. Panesi, "Bayesian machine learning approach to the quantification of uncertainties on *ab initio* potential energy surfaces," J. Phys. Chem. A **124**, 5129 (2020).
[93]M. Wen and E. B. Tadmor, "Uncertainty quantification in molecular simulations with dropout neural network potentials," npj Comput. Mater. **6**, 124 (2020).
[94]A. R. Tan, S. Urata, S. Goldman, J. C. B. Dietschreit, and R. Gómez-Bombarelli, "Single-model uncertainty quantification in neural network potentials does not consistently outperform model ensembles," npj Comput. Mater. **9**, 225 (2023).
[95]L. Zhang, D.-Y. Lin, H. Wang, R. Car, and E. Weinan, "Active learning of uniformly accurate interatomic potentials for materials simulation," Phys. Rev. Mater. **3**, 023804 (2019).
[96]J. S. Smith, B. Nebgen, N. Lubbers, O. Isayev, and A. E. Roitberg, "Less is more: Sampling chemical space with active learning," J. Chem. Phys. **148**, 241733 (2018).
[97]M. Jiang, T. Xing, E. Zio, and X. Zhu, "A Bayesian data-driven framework for aleatoric and epistemic uncertainty quantification in remaining useful life predictions," IEEE Sens. J. **24**, 42255 (2024).
[98]J. Dai, S. Adhikari, and M. Wen, "Uncertainty quantification and propagation in atomistic machine learning," Rev. Chem. Eng. **41**, 333 (2025).
[99]F. Grasselli, S. Chong, V. Kapil, S. Bonfanti, and K. Rossi, "Uncertainty in the era of machine learning for atomistic modeling," arXiv:2503.09196 (2025).
[100]T. Wang, Y. Wang, J. Zhou *et al.*, "From aleatoric to epistemic: Exploring uncertainty quantification techniques in artificial intelligence," arXiv:2501.03282 (2025).
[101]R. R. Rahaman, "Uncertainty quantification and deep ensembles," Adv. Neural Inf. Process. Syst. **34**, 20063 (2021).
[102]D. Althoff, L. N. Rodrigues, and H. C. Bazame, "Uncertainty quantification for hydrological models based on neural networks: The dropout ensemble," Stoch. Environ. Res. Risk Assess. **35**, 1051 (2021).
[103]A. Olivier, M. D. Shields, and L. Graham-Brady, "Bayesian neural networks for uncertainty quantification in data-driven materials modeling," Comput. Methods Appl. Mech. Eng. **386**, 114079 (2021).
[104]Y. Gal, P. Koumoutsakos, F. Lanusse, G. Louppe, C. Papadimitriou *et al.*, "Bayesian uncertainty quantification for machine-learned models in physics," Nat. Rev. Phys. **4**, 573 (2022).
[105]J. Zhang, "Modern Monte Carlo methods for efficient uncertainty quantification and propagation: A survey," Wiley Interdiscip. Rev. Comput. Stat. **13**, e1539 (2021).
[106]H.-R. Bae, R. V. Grandhi, and R. A. Canfield, "Epistemic uncertainty quantification techniques including evidence theory for large-scale structures," Comput. Struct. **82**, 1101 (2004).
[107]P. Manfredi and R. Trinchero, "A probabilistic machine learning approach for the uncertainty quantification of electronic circuits based on Gaussian process regression," IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. **41**, 2638 (2022).
[108]R. Hart, L. Yu, Y. Lou *et al.*, "Improvements on uncertainty quantification for node classification via distance based regularization," Adv. Neural Inf. Process. Syst. **36**, 55454 (2023).
[109]S. Munikoti, D. Agarwal, L. Das, B. Natarajan *et al.*, "A general framework for quantifying aleatoric and epistemic uncertainty in graph neural networks," Neurocomputing **521**, 1 (2023).
[110]V. Kuleshov, N. Fenner, and S. Ermon, "Accurate uncertainties for deep learning using calibrated regression," in *ICML* (PMLR, 2018).
[111]P. Pernot, "Calibration in machine learning uncertainty quantification: Beyond consistency to target adaptivity," APL Mach. Learn. **1**, 046121 (2023).
[112]Y. Liu, L. Wang, W. Zhu, and X. Yu, "Farthest point sampling in property designated chemical feature space as an effective strategy for enhancing the machine learning model performance for small scale chemical dataset," J. Mater. Inform. **5**, 39 (2025).
[113]A. P. Bartók, J. Kermode, N. Bernstein, and G. Csányi, "Machine learning a general-purpose interatomic potential for silicon," Phys. Rev. X **8**, 041048 (2018).
[114]A. D. Naghdi, F. Pellegrini, E. Küçükbenli *et al.*, "Neural network interatomic potentials for open surface nano-mechanics applications," Acta Mater. **277**, 120200 (2024).
[115]D. E. Smirnova, A. Y. Kuksin, S. V. Starikov, V. V. Stegailov, Z. Insepov, J. Rest, and A. M. Yacout, "A ternary EAM interatomic potential for U–Mo alloys with xenon," Modell. Simul. Mater. Sci. Eng. **21**, 035011 (2013).
[116]J. Byggmästar, K. Nordlund, and F. Djurabekova, "Gaussian approximation potentials for body-centered-cubic transition metals," Phys. Rev. Mater. **4**, 093802 (2020).
[117]S. Plimpton, "Fast parallel algorithms for short-range molecular dynamics," J. Comput. Phys. **117**, 1 (1995).
[118]A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, and S. J. Plimpton, "LAMMPS—A flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales," Comput. Phys. Commun. **271**, 108171 (2022).
[119]J. Tersoff, "Empirical interatomic potential for carbon, with applications to amorphous carbon," Phys. Rev. Lett. **61**, 2879 (1988).
[120]J. Tersoff, "Modeling solid-state chemistry: Interatomic potentials for multicomponent systems," Phys. Rev. B **39**, 5566 (1989).
[121]T. Werder, J. H. Walther, R. L. Jaffe, T. Halicioglu, and P. Koumoutsakos, "On the water-carbon interaction for use in molecular dynamics simulations of graphite and carbon nanotubes," J. Phys. Chem. B **107**, 1345 (2003).
[122]G. Kresse and J. Furthmüller, "Efficient iterative schemes for *ab initio* total-energy calculations using a plane-wave basis set," Phys. Rev. B **54**, 11169 (1996).
[123]P. E. Blöchl, O. Jepsen, and O. K. Andersen, "Improved tetrahedron method for Brillouin-zone integrations," Phys. Rev. B **49**, 16223 (1994).
[124]B. Hammer, L. B. Hansen, and J. K. Nørskov, "Improved adsorption energetics within density-functional theory using revised Perdew-Burke-Ernzerhof functionals," Phys. Rev. B **59**, 7413 (1999).
[125]S. Grimme, S. Ehrlich, and L. Goerigk, "Effect of the damping function in dispersion corrected density functional theory," J. Comput. Chem. **32**, 1456 (2011).
[126]M. R. Uhlig, D. Martin-Jimenez, and R. Garcia, "Atomic-scale mapping of hydrophobic layers on graphene and few-layer MoS_2 and WSe_2 in water," Nat. Commun. **10**, 2606 (2019).
[127]D. Kim, D. S. King, P. Zhong, and B. Cheng, "Learning charges and long-range interactions from energies and forces," arXiv:2412.15455 (2024).
[128]C. Zhang, M. C. Andrade, Z. K. Goldsmith, A. S. Raman, Y. Li, P. Piaggi, X. Wu, R. Car, and A. Selloni, "Electrical double layer and capacitance of TiO_2 electrolyte interfaces from first principles simulations," arXiv:2404.00167 (2024).
[129]C. Zhang, M. F. Calegari Andrade, Z. K. Goldsmith, A. S. Raman, Y. Li, P. M. Piaggi, X. Wu, R. Car, and A. Selloni, "Molecular-scale insights into the electrical double layer at oxide-electrolyte interfaces," Nat. Commun. **15**, 10270 (2024).
[130]Y. Xiao, Y. Wang, S.-H. Bo, J. C. Kim, L. J. Miara, and G. Ceder, "Understanding interface stability in solid-state batteries," Nat. Rev. Mater. **5**, 105 (2020).
[131]X. Wang, X. Wu, P. Ying, Z. Fan, and H. Sun, "Interface phonon modes governing the ideal limit of thermal transport across diamond/cubic boron nitride interfaces," arXiv:2504.18473 (2025).

08 February 2026 12:36:01

© Author(s) 2026

[132]T. Huang, W. Dong, F. Wu, X. Li, G. Shi *et al.*, "Uncertainty-driven knowledge distillation for language model compression," IEEE/ACM Trans. Audio Speech Lang. Process. **31**, 2850 (2023).
[133]K. Song, R. Zhao, J. Liu *et al.*, "General-purpose machine-learned potential for 16 elemental metals and their alloys," Nat. Commun. **15**, 10208 (2024).
[134]A. Siddiqui and N. D. M. Hine, "Machine-learned interatomic potentials for transition metal dichalcogenide $Mo_{1-x}W_xS_{2-2y}Se_{2y}$ alloys," npj Comput. Mater. **10**, 169 (2024).
[135]Y. Chiang, T. Kreiman, E. Weaver *et al.*, "MLIP Arena: Advancing fairness and transparency in machine learning interatomic potentials through an open and accessible benchmark platform," in *AI for Accelerated Materials Design* (ICLR, 2025).
[136]F. Ekström Kelvinius, D. Georgiev, A. Toshev *et al.*, "Accelerating molecular graph neural networks via knowledge distillation," in *Advances in Neural Information Processing Systems* (Curran Associates, Inc., 2023), Vol. 36.
[137]I. Amin, S. Raja, and A. Krishnapriyan, "Towards fast, specialized machine learning force fields: Distilling foundation models via energy Hessians," arXiv:2501.09009 (2025).
[138]K. Schütt, O. Unke, and M. Gastegger, "Equivariant message passing for the prediction of tensorial properties and molecular spectra," in *ICML* (PMLR, 2021).
[139]M. J. Willatt, F. Musil, and M. Ceriotti, "Feature optimization for atomistic machine learning yields a data-driven construction of the periodic table of the elements," Phys. Chem. Chem. Phys. **20**, 29661 (2018).
[140]E. Wigner, *Group Theory and Its Application to the Quantum Mechanics of Atomic Spectra* (Elsevier, 2012), Vol. 5.
[141]Y. Wang (2023). "MACE-FT Tutorial," Github repository. https://github.com/YangshuaiWang/mace-ft-tutorial.

08 February 2026 12:36:01

© Author(s) 2026